

OpenClaw and Ollama in Agentic AI: Toward Fully Autonomous and Scalable AI Agent Systems

Konstantinos I. Roumeliotis^{1*} and Ranjan Sapkota^{2*}

^{1*}Department of Informatics and Telecommunications, University of the Peloponnese, Tripoli, 22131, Greece.

^{2*}Department of Biological and Environmental Engineering, Cornell University, Ithaca, NY, 14850, USA.

*Corresponding author(s). E-mail(s): k.roumeliotis@uop.gr; rs2672@cornell.edu;

Abstract

The rapid transition from reactive large language model (LLM) interfaces to persistent, action-capable systems has revealed fundamental gaps in the architectural understanding of Agentic AI, particularly in disentangling inference, orchestration, and execution layers. Despite significant progress, there remains a lack of unified frameworks that systematically explain how autonomous agents can be designed, deployed, and evaluated as full-stack systems. This paper presents a comprehensive, layered analysis of Agentic AI architectures. We begin by establishing the foundations of Agentic AI and layered architectures, examining the evolution from reactive LLMs to persistent agentic systems and identifying core design principles such as modularity, memory hierarchy, and continuous execution. We then analyze OpenClaw and Ollama as a full-stack Agentic AI architecture, where Ollama functions as the LLM inference layer and OpenClaw operates as the agent runtime layer, enabling seamless integration from model inference to autonomous reasoning, planning, and action. To substantiate this architectural perspective, we introduce a prototype experimental validation of the OpenClaw–Ollama full-stack system. Through controlled configurations, benchmark task design, and system-level evaluation metrics, the results demonstrate a consistent monotonic improvement in performance as architectural complexity increases, confirming that autonomous capabilities such as tool use and persistent memory emerge from system integration rather than isolated models. Building on these findings, we further analyze operational challenges, safety, and evaluation, highlighting critical issues in security, privacy, governance, and benchmarking of persistent agentic systems. Finally, we explore future research directions, including scalable multi-agent systems, distributed architectures, and human-centered, responsible

autonomous AI. Overall, this work establishes a unified architectural framework for Agentic AI, provides empirical validation of full-stack autonomous systems, and outlines a roadmap toward scalable, secure, and trustworthy autonomous agents. Prototype experimental validation, architecture, models, code, and experimental datasets are openly released to support reproducibility, transparency, and community benchmarking at [GitHub](#).

Keywords: Autonomous AI Agents, Agentic AI, AI Agents, Open-Source Agents, Artificial Intelligence

1 Introduction

The emergence of large language models (LLMs) has fundamentally reshaped the trajectory of artificial intelligence, shifting the field from narrow task-specific pipelines toward increasingly general-purpose reasoning systems capable of language understanding, code generation, multimodal interpretation, and decision support [1, 2]. Early deployments of LLMs, however, were predominantly reactive: a user issued a prompt, the model generated a response, and the interaction terminated unless a new prompt was manually provided [3]. Such systems demonstrated remarkable linguistic competence, yet they remained limited in persistence, autonomy, memory continuity, and grounded action [4]. As a result, a growing gap has become evident between what foundation models can express in natural language and what real-world intelligent systems must do in practice, namely perceive changing environments, maintain long-term objectives, coordinate tools, preserve state across interactions, and act over extended temporal horizons. This gap has catalyzed the rise of **Agentic AI**, a paradigm in which LLMs serve not merely as text generators but as cognitive engines embedded within broader software architectures for planning, memory management, tool use, and autonomous execution [5, 6].

Within this transition, the central challenge is no longer only model capability, but system architecture. In other words, the practical realization of autonomous AI agents depends on how inference engines, memory modules, orchestration layers, execution pipelines, communication interfaces, and safety controls are composed into an integrated runtime [7, 8]. A single powerful model is insufficient to produce a robust agentic system if it lacks persistent state, reliable access to external tools, governance constraints, or mechanisms for long-horizon reasoning [9, 10]. Consequently, the engineering focus of contemporary AI research is moving toward layered architectures that separate the concerns of model inference, agent orchestration, and environment interaction [7, 11]. This architectural shift has become especially important for LLM-based multi-agent systems, in which multiple AI agents may collaborate, specialize, exchange information, and jointly execute complex workflows [12, 13]. In such settings, modularity, interoperability, and runtime control become essential design goals.

Two emerging technologies illustrate this layered transformation particularly well: **Ollama** ([Source Link](#)) and **OpenClaw** ([Source Link](#)). Ollama represents the **LLM inference layer**, providing a lightweight and practical mechanism for running large

language models locally, efficiently, and with stronger privacy control [14]. It addresses a major limitation of cloud-dependent LLM deployments by enabling on-device or self-hosted inference, thus improving deployment flexibility, latency characteristics, and data sovereignty [15]. OpenClaw, by contrast, represents the **agent runtime layer**, offering an architectural framework in which models can be embedded into persistent, tool-using, memory-enabled, and action-capable autonomous agents [16, 17]. Its significance lies not in model training, but in its orchestration of cognition, memory, scheduling, and execution into a continuous operational loop [18, 19]. When considered together, Ollama and OpenClaw provide a useful lens through which to examine the full-stack realization of **Agentic AI**: one supplies the computational substrate for inference, while the other organizes that intelligence into persistent agent behavior.

This layered perspective is scientifically important for several reasons. First, it clarifies that modern Autonomous Agents are not monolithic artifacts but composite systems, and that their behavior emerges from interactions among heterogeneous modules rather than from model weights alone [20, 21]. Second, it enables a more rigorous taxonomy of agentic architectures by distinguishing between the inference layer, the runtime layer, the memory layer, the tool interaction layer, and the governance layer [22]. Third, it provides a clearer basis for analyzing system-level trade-offs, such as privacy versus scalability [23, 24], local deployment versus cloud elasticity [25, 26], architectural flexibility versus integration complexity, and autonomy versus controllability. Finally, it makes it possible to evaluate systems such as OpenClaw and Ollama not merely as isolated tools, but as representative components of a broader infrastructural stack for Agentic AI, AI agents, and LLM-based multi-agent systems with autonomous AI agents.

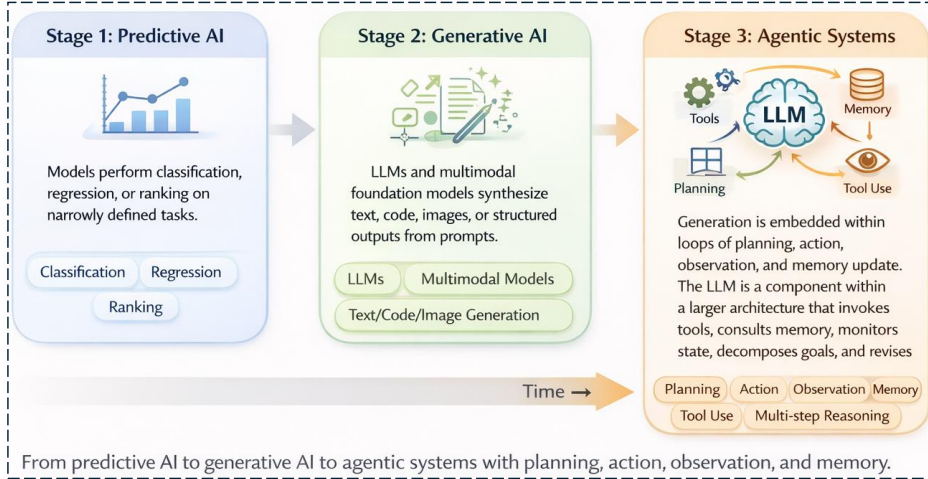


Fig. 1 A structured evolution of artificial intelligence from predictive models to generative systems and ultimately to agentic architectures integrating planning, action, observation, and memory within autonomous decision-making loops.

As explained in Figure 1, from a historical perspective, the path toward Agentic AI can be understood as a progression through several stages. The first stage emphasized predictive AI, where models performed classification, regression, or ranking on narrowly defined tasks. The second stage introduced generative AI, in which LLMs and multimodal foundation models could synthesize text, code, images, or structured outputs from prompts. The third stage, now rapidly expanding, concerns agentic systems, where generation is embedded within loops of planning, action, observation, and memory update. In this third stage, an LLM is not the entire system, but a component within a larger architecture that may invoke tools, consult memory stores, monitor external state, decompose goals into subtasks, and continuously revise its own strategy. This transition is particularly relevant for scientific, enterprise, robotic, and cyber-physical applications, where decision continuity, contextual awareness, and system-level reliability are indispensable.

Despite substantial progress, the literature remains fragmented. Existing studies frequently examine agent frameworks [27, 28], autonomous AI assistants [29–32], orchestration libraries [33–35], local inference engines [36–38], or tool-use paradigms in isolation [39]. Some works focus on LLM reasoning and planning [40, 41], others on memory augmentation [42, 43], retrieval [44, 45], or orchestration [46, 47], while still others emphasize privacy-preserving local deployment [48, 49]. Yet there is a clear shortage of review-level analyses that jointly study how an inference framework such as **Ollama** and an agent runtime such as **OpenClaw** together instantiate a coherent **Agentic AI Architecture**. This gap is nontrivial. Without a unified architectural perspective, it becomes difficult to answer foundational questions such as: **What precisely differentiates inference from agency? Why are both layers needed in a practical autonomous system? How should persistent agent runtimes be evaluated when they depend on external model engines? What new safety and governance risks emerge when local inference and autonomous execution are combined? How should future infrastructures evolve toward scalable, trustworthy, and interoperable multi-agent ecosystems?**

The case for studying Ollama is especially strong in the current technological landscape. As concerns about privacy, data governance, regulatory compliance, model cost, and latency continue to grow, local LLM inference has become increasingly attractive. Self-hosted inference engines reduce dependence on external APIs, lower recurring costs in some deployment scenarios, and support offline or edge-based operation [50–52]. These properties are particularly relevant in domains involving sensitive user data, proprietary enterprise workflows, or constrained connectivity. However, local inference alone does not create an autonomous agent. It provides token generation, not persistent behavior; prediction, not orchestration; language output, not tool-governed action [53, 54]. Thus, an inference engine such as Ollama should be understood as a necessary but insufficient component of a complete Agentic AI system.

Similarly, the case for studying OpenClaw arises from the growing need for persistent, tool-using, memory-enabled AI runtimes. In contrast to ordinary chat interfaces, OpenClaw-type systems aim to maintain state over time, manage long-term goals, invoke external skills, and operate across multiple channels and contexts [19, 55, 56]. This aligns closely with the defining properties of autonomous AI agents: persistence,

modular actionability, memory continuity, and the ability to reason across multi-step workflows [7, 57, 58]. Yet such agent runtimes also introduce new forms of risk. When an LLM is placed inside a persistent execution framework with access to files, APIs, communication tools, or schedulers, the system’s failure modes extend far beyond hallucinated text [59]. Risks now include memory poisoning, malicious tool invocation, privacy leakage, prompt injection, unsafe automation, and loss of human oversight [60, 61]. Therefore, OpenClaw is not only architecturally significant but also a compelling case study for examining safety, trust, and governance in full-stack agentic systems.

A full understanding of OpenClaw and Ollama also requires attention to layered architectures. In conventional software engineering, layered design promotes modularity, abstraction, and maintainability by separating system concerns into relatively independent components [62, 63]. A similar principle applies to Agentic AI. The inference layer is responsible for model execution and token generation [64, 65]. The runtime layer is responsible for orchestrating agent reasoning loops, memory access, and tool calls [7, 8]. The execution layer interfaces with external systems such as browsers, filesystems, APIs, and communication channels [66]. The governance layer constrains, monitors, logs, and audits behavior [67, 68]. The memory layer preserves continuity across sessions and tasks [69, 70]. Distinguishing among these layers is essential for both analysis and design, because the capabilities, bottlenecks, and risks of each layer differ substantially. For example, increasing model size may improve reasoning quality but worsen latency; adding more tools may expand agent utility but enlarge the attack surface; local inference may improve privacy but impose hardware limits. A scientifically rigorous review must therefore consider the entire architecture, rather than conflating all system behavior with the LLM itself.

The importance of this layered view is magnified in the context of LLM-based multi-agent systems. Multi-agent architectures introduce additional complexity, including inter-agent communication, role specialization, distributed memory, coordination protocols, negotiation mechanisms, and conflict resolution strategies [71]. In such systems, the inference layer may be centralized or distributed, the runtime layer may manage multiple heterogeneous agents, and the governance layer must handle both individual and collective behavior [72, 73]. OpenClaw and Ollama, while often discussed in single-agent deployment contexts, provide conceptual building blocks that can be extended toward broader agent ecosystems. Ollama can support local inference for multiple agents or specialized local models [74, 75], while OpenClaw-like runtimes can coordinate persistent agent roles, memories, and tools [16, 76, 77]. This creates a fertile research space at the intersection of Agentic AI, multi-agent systems, AI agents, and trustworthy autonomous infrastructures.

Another major motivation for this review lies in evaluation. Traditional model benchmarks are insufficient for full-stack agentic systems because they do not capture persistence, tool-use reliability, long-horizon planning, recovery behavior, or governance compliance [78, 79]. A model may perform well on reasoning benchmarks yet fail catastrophically when embedded in a continuous execution environment [80–82]. Conversely, a modest model may produce useful agentic behavior when coupled with

robust memory management, careful tool abstraction, and strong oversight mechanisms [83, 84]. This indicates that the unit of analysis must shift from isolated model outputs to system-level performance. Accordingly, any meaningful review of OpenClaw and Ollama must address not only architecture, but also security, privacy, governance, evaluation metrics, benchmarking challenges, and future research roadmaps.

Against this background, this paper aims to provide a comprehensive and architecturally grounded review of **OpenClaw and Ollama in Agentic AI**. Rather than treating these technologies as disconnected software tools, the paper conceptualizes them as complementary layers within a unified full-stack system for **Autonomous Agent Systems**. Specifically, the review synthesizes the conceptual foundations of agentic architectures, examines the role of Ollama as a local LLM inference layer, analyzes OpenClaw as a persistent agent runtime layer, studies the integration of inference and orchestration into autonomous agent pipelines, evaluates operational challenges in safety and governance, and identifies future directions for scalable and responsible multi-agent ecosystems. In doing so, the paper seeks to bridge the gap between model-centric AI discourse and architecture-centric autonomous systems research.

To provide a clear analytical flow, the review is structured around four major sections as illustrated in Figure 2. The first section, *Foundations of Agentic AI and Layered Architectures*, establishes the conceptual basis for the paper by tracing the evolution from reactive LLMs to layered agentic systems and by identifying the key design principles of autonomous agent architectures. The second section, *OpenClaw and Ollama: A Full-Stack Agentic AI Architecture*, presents the core technical analysis, examining Ollama as the inference layer, OpenClaw as the runtime layer, and their integration into complete autonomous agent pipelines. The third section, *Operational Challenges, Safety, and Evaluation*, addresses the critical issues that arise in practice, including security, privacy, governance, trust, and benchmarking of full-stack agentic systems. The fourth section, *Future Directions and Research Opportunities*, expands the perspective toward multi-agent, distributed, and human-centered futures, highlighting research gaps and pathways toward more scalable, trustworthy, and interoperable architectures.

To make these objectives explicit, this review is guided by the following research questions:

1. RQ1: Foundations of Agentic AI and Layered Architectures: How have reactive LLM systems evolved into layered Agentic AI architectures, and what foundational design principles define robust autonomous AI agents and LLM-based multi-agent systems?
2. RQ2: OpenClaw and Ollama as a Full-Stack Agentic AI Architecture: How do Ollama and OpenClaw function as complementary inference and runtime layers, and what architectural insights emerge when they are analyzed together as a complete system for Autonomous Agent Systems?
3. RQ3: Operational Challenges, Safety, and Evaluation: What are the principal security, privacy, governance, reliability, and benchmarking challenges associated with layered Agentic AI Architecture, especially when local inference and persistent autonomous execution are combined?



Fig. 2 Graphical overview of the analytical flow of this review paper. Beginning with the introduction, the study progresses through four interconnected stages: (i) foundational concepts of layered Agentic AI, (ii) architectural integration of Ollama and OpenClaw as a full-stack system, (iii) prototype experimental validation and system-level analysis, and (iv) operational challenges and future directions toward scalable, human-centered, and trustworthy autonomous agent systems.

4. RQ4: Future Directions and Research Opportunities: What future developments are most critical for advancing scalable, interoperable, human-centered, and trustworthy AI agents, including next-generation LLM-based multi-agent systems with autonomous AI agents?

Figure 2 summarizes the conceptual flow of this review and visually links the introduction to the four core analytical sections that structure the remainder of the paper.

2 Foundations of Agentic AI and Layered Architectures

2.1 From Reactive LLMs to Layered Agentic AI Systems

As depicted in Figure 3, the evolution of artificial intelligence systems from reactive large LLM interfaces to fully operational Agentic AI systems represents a fundamental paradigm shift in how intelligence is conceptualized, deployed, and evaluated. Traditional LLM applications are inherently stateless and reactive, operating under a prompt-response paradigm in which each interaction is largely independent of previous context unless explicitly provided [85, 86]. While such systems demonstrate strong capabilities in natural language understanding, reasoning, and generation, they lack essential properties required for real-world autonomy, including persistence, long-term memory, goal continuity, and the ability to act upon external environments [87, 88]. Consequently, these systems are limited in handling complex, multi-step tasks that

require iterative reasoning, dynamic adaptation, and continuous interaction with tools or environments [12, 89].

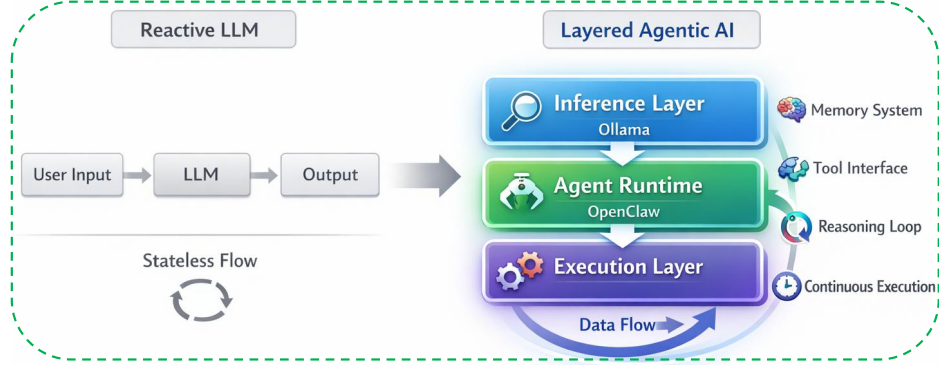


Fig. 3 Illustrating the stateless Reactive LLM pipelines with layered Agentic AI systems. The left side shows a Stateless Flow where input is directly mapped to output without memory or action. The right side presents a structured architecture with Inference Layer, Agent Runtime, and Execution Layer, enabling persistent behavior. Key components such as Memory System, Tool Interface, Reasoning Loop, and Continuous Execution support autonomous operation. Data Flow and cyclic feedback illustrate iterative reasoning, highlighting the transition from passive models to fully integrated autonomous agent systems.

In contrast, autonomous agents extend beyond passive response generation by incorporating mechanisms for reasoning, memory, planning, and action within a continuous operational loop [90]. These systems are capable of maintaining internal state, tracking objectives over time, and interacting with external resources such as APIs, databases, and physical or virtual tools [7, 91]. This transition from reactive models to persistent agents is not merely an incremental improvement but a structural transformation in system design. It reflects a shift from model-centric intelligence toward architecture-centric intelligence, where the behavior of the system emerges from the integration of multiple components rather than from the LLM alone [92, 93].

A key enabler of this transformation is the introduction of layered AI architecture, in which complex systems are decomposed into distinct but interconnected layers, each responsible for specific functionalities [11]. At a minimum, modern Agentic AI architecture can be conceptualized as consisting of three primary layers as shown in Figure 3: (i) the LLM inference layer, responsible for language understanding and generation; (ii) the agent orchestration or runtime layer, responsible for reasoning, planning, and coordinating actions; and (iii) the execution layer, responsible for interacting with external tools, environments, and data sources. This separation of concerns is critical for achieving modularity, scalability, and maintainability in autonomous systems.

Within this layered architectural framework, systems such as Ollama and OpenClaw can be interpreted as representative components operating across distinct strata of the Agentic AI architecture. Specifically, Ollama functions at the LLM inference layer, providing efficient, local, and privacy-preserving model execution capabilities

[14, 48]. By abstracting model loading, serving, and interaction processes, it enables flexible deployment across local and edge environments [94]. However, Ollama primarily operates as a stateless inference engine and does not inherently support persistence, memory management, or autonomous decision-making [20].

These higher-level capabilities are instead realized at the agent runtime layer, exemplified by OpenClaw, which orchestrates iterative reasoning loops, manages long-term memory, enables tool invocation, and supports continuous task execution [95]. The separation between inference and runtime layers allows independent optimization of system components: inference engines can focus on computational efficiency and model accuracy, while runtime systems are designed to enhance autonomy, planning, and interaction within autonomous agent systems [96, 97].

Several enabling paradigms underpin the functionality of persistent agent systems. One of the most important is the reasoning-action loop [98, 99], often implemented through iterative cycles of thought, action, observation, and memory update [100]. This loop allows agents to decompose complex goals into sub-tasks, evaluate intermediate outcomes, and refine their strategies dynamically [10]. Closely related is the concept of tool calling, where agents invoke external functions or services to extend their capabilities beyond text generation [101, 102]. This includes web browsing [103, 104], code execution [105, 106], database querying [107, 108], and interaction with domain-specific APIs [109, 110].

Additionally, memory retrieval mechanisms enable agents to access historical context, past decisions, and accumulated knowledge, thereby overcoming the limitations of fixed context windows in LLMs [111]. Finally, continuous execution mechanisms, such as scheduling or event-driven triggers, allow agents to operate proactively rather than solely in response to user input [112, 113].

Despite their success, stateless LLM systems exhibit several fundamental limitations that motivate the adoption of layered architectures. First, the absence of persistent memory restricts their ability to maintain continuity across interactions, leading to fragmented reasoning and repeated context reconstruction. Second, their reactive nature prevents them from initiating actions or monitoring environments autonomously. Third, their limited integration with external systems constrains their practical utility in real-world workflows. These limitations become particularly pronounced in applications requiring long-horizon planning, multi-step reasoning, or coordination across multiple tools and data sources.

Layered Agentic AI systems address these limitations by decoupling inference from orchestration and execution [9]. This modular design enables independent scaling and optimization of each layer, facilitating more robust and adaptable systems [8, 114]. For example, improvements in the LLM inference layer can enhance reasoning quality without altering the runtime logic, while enhancements in the agent runtime can improve planning and execution without requiring retraining of the underlying model [115, 116]. Moreover, layered architectures provide a clearer framework for implementing safety mechanisms, monitoring system behavior, and enforcing governance policies across different components [117, 118]. Table 1 shows the evolution summary from reactive to layered LLM-based agentic AI systems.

Table 1 Evolution from Reactive LLM Systems to Layered Agentic AI Architectures

System Characteristic	Reactive LLM Systems	Layered Agentic AI Systems
Interaction Paradigm	Prompt-response, user-driven interaction	Continuous, goal-driven interaction with autonomous execution
State Management	Stateless or session-limited context	Persistent memory with long-term context retention
Architecture Design	Monolithic LLM-centric pipeline	Layered architecture (Inference, Runtime, Execution)
Reasoning Capability	Single-step reasoning per prompt	Multi-step reasoning with iterative refinement loops
Execution Ability	No direct action capability	Tool-based action execution in external environments
Memory Utilization	Limited to context window	Hierarchical memory (short-term, long-term, episodic)
Autonomy Level	Fully reactive	Semi-autonomous to fully autonomous agents
Tool Integration	Minimal or absent	Modular tool interfaces with API/function calling
Adaptability	Static behavior per prompt	Dynamic planning and adaptation over time
System Triggering	User-initiated only	Event-driven and time-triggered execution
Scalability	Limited by model and prompt size	Scalable across agents, tools, and distributed systems
Deployment Mode	Primarily cloud-based APIs	Local, cloud, or hybrid (e.g., Ollama + OpenClaw)
Control Flow	Linear input-output mapping	Cyclic reasoning-action-feedback loops
System Intelligence	Model-centric intelligence	Architecture-centric intelligence

2.2 Design Principles of Autonomous Agents Architecture

Robust autonomous agents architecture is fundamentally governed by a set of system-level design principles as shown in Figure 4, that extend beyond model capability and into the domain of structured, modular, and controllable AI systems engineering. Unlike monolithic LLM applications, modern Agentic AI systems are inherently composite, integrating multiple interacting subsystems including inference engines, reasoning modules, memory layers, execution pipelines, and governance mechanisms [7, 119]. The effectiveness of such systems depends critically on how these components are architected, coordinated, and optimized under real-world constraints such as latency, scalability, reliability, and safety [120, 121].

One of the most essential principles is modularity, which enables the decomposition of agent systems into independent but interoperable components [122]. Modular design allows different layers such as the LLM inference layer, the agent runtime layer, and the execution interface to evolve independently without tightly coupling system behavior to a single implementation [123, 124]. For example, separating inference (e.g., Ollama) from orchestration (e.g., OpenClaw) enables flexible deployment strategies, where models can be swapped, upgraded, or distributed without redesigning the entire agent pipeline. This abstraction is particularly important in modular AI systems, where heterogeneous components must interoperate across diverse environments and tasks.

Closely related is the principle of persistence, which distinguishes autonomous agents from transient systems [124, 125]. Persistent agents maintain internal state across

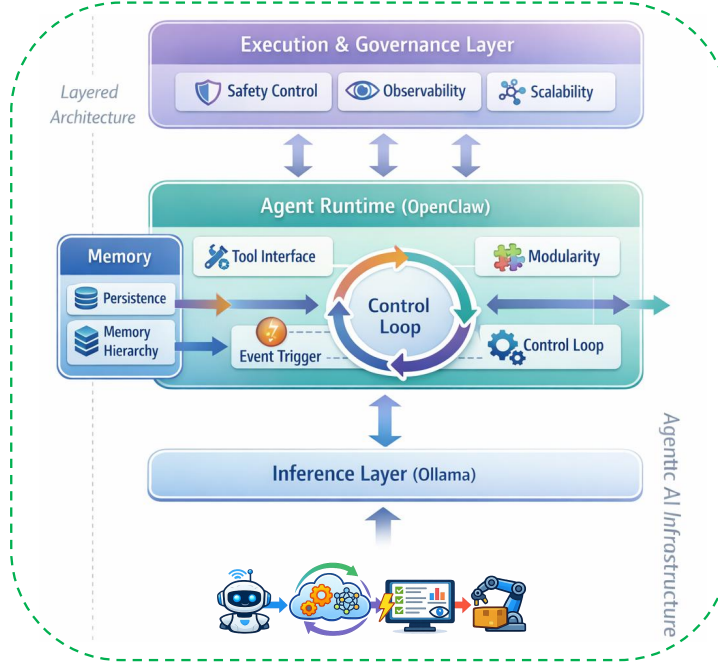


Fig. 4 Showing the layered design of autonomous agents architecture, integrating inference (Ollama), runtime (OpenClaw), memory, tools, and governance. Core principles Modularity, Persistence, Memory Hierarchy, Tool Interface, Observability, Safety Control, and Scalability are embedded across layers. Control Loop and Event Trigger mechanisms enable dynamic and continuous execution, while structured Data Flow ensures coordination among components. The diagram emphasizes how layered Agentic AI infrastructure supports reliable, scalable, and secure operation of persistent autonomous agents in real-world environments.

interactions, enabling long-term goal tracking, contextual continuity, and adaptive behavior [126, 127]. This persistence is supported by a structured memory hierarchy, which may include short-term context buffers, long-term storage, episodic memory, and semantic knowledge representations [128]. A well-designed persistent memory architecture ensures that relevant information is retained, retrieved, and updated efficiently, allowing the agent to reason over extended temporal horizons while avoiding redundancy or memory saturation [129].

Another critical design element is tool abstraction, which enables agents to extend their capabilities beyond text generation [130, 131]. Tools are typically encapsulated as callable interfaces that allow the agent to interact with external systems such as APIs, databases, file systems, or computational engines [20]. Effective agent system design requires a clear separation between decision-making logic and tool execution, ensuring that tools are invoked in a controlled, interpretable, and verifiable manner [132, 133]. This abstraction also facilitates extensibility, as new tools can be integrated without modifying core reasoning logic.

Observability is equally important in autonomous systems, as it provides visibility into internal states, decision processes, and execution outcomes [132, 134]. Observability

mechanisms include logging, tracing, state inspection, and performance monitoring, all of which are essential for debugging, auditing, and optimizing agent behavior [135, 136]. In the context of Agentic AI infrastructure, observability also supports transparency and trust, enabling users and developers to understand how decisions are made and how actions are executed over time [8, 137].

To ensure safe and reliable operation, safety constraints must be embedded at multiple layers of the architecture. These constraints may include access control policies, sandboxed execution environments, validation checks for tool outputs, and guardrails for reasoning processes [138, 139]. In persistent systems, safety is not a static property but a dynamic requirement that must be enforced continuously as the agent interacts with evolving environments [140, 141]. This necessitates integration of governance mechanisms within the architecture rather than as external add-ons.

Scalability is another key consideration, particularly as agentic systems evolve toward multi-agent and distributed configurations [142, 143]. Scalability involves both horizontal expansion (e.g., multiple agents or distributed inference nodes) and vertical optimization (e.g., efficient memory usage and low-latency execution) [36, 144]. Architectural decisions such as centralized versus distributed control, and cloud-based versus local inference, directly impact scalability [145, 146]. For instance, centralized architectures simplify coordination but may introduce bottlenecks, whereas distributed architectures enhance robustness and parallelism but require sophisticated synchronization mechanisms [147, 148].

Control flow within autonomous agents is typically governed by control loops, which define iterative cycles of reasoning, action, observation, and state update [10, 149]. These loops may operate in conjunction with event-driven execution, where agents respond to external triggers, and time-triggered autonomy, where actions are initiated based on scheduled intervals [150, 151]. Effective coordination of these mechanisms requires state synchronization across components, ensuring consistency between memory, reasoning state, and external interactions. Without proper synchronization, agents may exhibit inconsistent or erroneous behavior due to stale or conflicting information [73, 152].

The separation between inference and orchestration layers further enhances system robustness by isolating concerns. The LLM inference layer can be optimized for computational efficiency, model accuracy, and hardware utilization, while the agent runtime layer focuses on decision-making logic, memory management, and execution coordination [36, 115]. This separation also enables hybrid deployment strategies, where inference may occur locally (e.g., via Ollama) while orchestration may be distributed across services or devices.

In summary, as explained in Table 2, the design of autonomous agents systems requires a holistic architectural perspective that integrates modularity, persistence, memory hierarchy, tool abstraction, observability, safety, and scalability. These principles collectively define the foundation of modern Agentic AI infrastructure, enabling systems that are not only intelligent but also reliable, adaptable, and controllable. This conceptual framework is essential for analyzing system-level trade-offs and guiding the development of next-generation agentic architectures.

Table 2 Core Design Principles and System-Level Roles in Autonomous Agents Architecture

Principle	Definition	Architectural Role	Implementation in Agentic AI Systems
Modularity	Decomposition of systems into independent, interoperable components	Enables separation of inference, runtime, memory, and execution layers for flexibility and maintainability	Layered architecture with Ollama (inference) and OpenClaw (runtime) decoupled; plug-and-play tool modules
Persistence	Ability to maintain state across interactions and time horizons	Supports long-term reasoning, goal tracking, and contextual continuity in autonomous agents	External memory stores, session tracking, and task history integration within runtime layer
Memory Hierarchy	Structured organization of short-term and long-term memory	Facilitates efficient retrieval, context compression, and knowledge accumulation	Context buffers, vector databases, episodic logs, and semantic memory modules
Tool Abstraction	Encapsulation of external functionalities as callable interfaces	Extends agent capability beyond text generation to real-world interaction	API wrappers, function calling, plugin-based tools (e.g., file systems, web APIs)
Observability	Monitoring and tracing of internal states and execution flows	Ensures transparency, debugging capability, and system accountability	Logging pipelines, execution traces, telemetry dashboards, and reasoning logs
Safety Control	Enforcement of constraints to prevent unsafe or unintended actions	Maintains secure and trustworthy operation in dynamic environments	Sandboxed execution, access control policies, validation layers, and guardrails
Scalability	Ability to expand system capacity across tasks, agents, and infrastructure	Supports multi-agent systems, distributed execution, and workload scaling	Distributed runtimes, parallel agent execution, and scalable inference backends
Control Loop	Iterative reasoning-action-feedback execution cycle	Enables adaptive decision-making and continuous refinement of agent behavior	Thought-action-observation loops, feedback integration, and dynamic replanning
Event Trigger	External or internal signals initiating agent execution	Allows reactive and proactive system behavior based on environment or schedule	Event-driven pipelines, API triggers, and scheduled task execution (cron/heartbeat)
State Synchronization	Consistency of data across memory, reasoning, and execution modules	Prevents conflicts, stale context, and inconsistent agent behavior	Shared memory states, synchronization protocols, and consistency validation mechanisms

3 OpenClaw and Ollama: A Full-Stack Agentic AI Architecture

<https://github.com/Applied-AI-Research-Lab/OpenClaw-and-Ollama-in-Agentic-AI>

3.1 Ollama as the LLM Inference Layer

Ollama represents a critical advancement in LLM runtime systems, enabling efficient, local, and privacy-preserving execution of large language models [14]. As a local AI inference engine, Ollama abstracts the complexities of model loading, optimization, and serving into a streamlined interface that allows developers and researchers to deploy on-device AI models with minimal overhead [153, 154]. Unlike cloud-based inference APIs, Ollama operates directly on local hardware, providing deterministic control over model execution, reduced latency, and enhanced data privacy [155, 156].

This capability is particularly important in applications where sensitive data cannot be transmitted to external servers, such as enterprise workflows, healthcare systems, or edge-based robotic platforms.

At its core, as depicted in Figure 5, the Ollama LLM framework provides a unified interface for managing multiple models, including open-source architectures such as LLaMA, Mistral, and other transformer-based systems [74]. It supports model versioning, efficient loading mechanisms, and optimized inference pipelines tailored to available hardware resources, including CPUs and GPUs [15]. Through its lightweight API interface, Ollama allows seamless integration with higher-level systems, enabling agent runtimes such as OpenClaw to invoke LLM capabilities as needed [22, 157]. This API-driven design is essential for modular AI inference architecture, where inference engines operate as independent services within a larger system.

From a systems perspective, Ollama functions as the foundational LLM inference layer within the broader Agentic AI architecture [158]. It is responsible for generating context-aware outputs based on input prompts, leveraging pre-trained model weights and token-level processing [159, 160]. However, it does not inherently maintain state, perform long-term planning, or execute actions [161, 162]. Instead, it operates as a stateless computational engine, producing outputs that must be interpreted and orchestrated by higher-level components [153, 163]. This distinction is crucial: while Ollama provides the cognitive capability of language understanding and reasoning, it does not constitute an autonomous system on its own.

The advantages of local inference through Ollama are explained in Figure 5. First, it eliminates reliance on external cloud services, reducing operational costs and mitigating risks associated with network latency and service availability. Second, it enhances **data security** by ensuring that sensitive inputs remain within the local environment. Third, it provides **deterministic performance control**, allowing developers to optimize resource allocation and execution parameters based on specific hardware configurations. These properties make Ollama particularly suitable for edge computing scenarios and environments requiring strict compliance with data governance regulations.

Despite these advantages, Ollama also presents several limitations. The most significant constraint is hardware dependency, as the performance and scalability of local inference are bounded by available computational resources. Large models may require substantial memory and processing power, limiting their applicability on resource-constrained devices [164–166]. Additionally, Ollama does not include built-in mechanisms for memory management, task planning, or tool orchestration, which are essential for autonomous behavior [167, 168]. As a result, it must be integrated with higher-level systems to achieve full Agentic AI functionality.

In summary, as illustrated in Figure 5, Ollama serves as a foundational component of AI inference architecture, providing efficient and flexible on-device AI models that enable scalable and privacy-preserving deployments. Its role within the Agentic AI systems stack is to supply the reasoning and language generation capabilities upon which autonomous behavior can be constructed [169–171]. When combined with agent runtimes such as OpenClaw, Ollama becomes a critical enabler of full-stack autonomous agents, bridging the gap between raw model inference and structured, persistent intelligence [16].

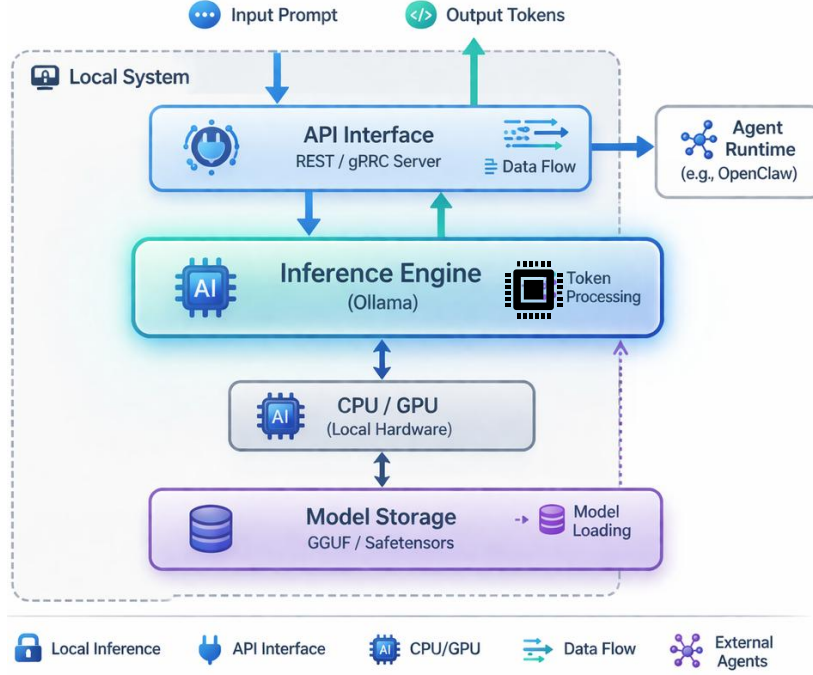


Fig. 5 Illustrating a local inference architecture where Model Storage hosts on-device models accessed by the Inference Engine (Ollama) through efficient Token Processing. The API Interface enables communication with external systems and External Agents, while Data Flow arrows depict prompt-to-output transformation. Hardware support via CPU/GPU ensures optimized execution. The Local Inference setup enhances privacy and latency control, positioning Ollama as a foundational component in scalable Agentic AI systems.

3.2 OpenClaw as the Agent Runtime Layer

OpenClaw represents a higher-level Agentic AI runtime that transforms stateless LLM outputs into persistent, goal-oriented, and action-capable autonomous agents [76, 172]. Unlike inference engines, OpenClaw is designed as a continuous execution framework that integrates reasoning, memory, tool usage, and scheduling into a unified system [172]. Its architecture is centered around the concept of persistent autonomy, where agents operate over extended time horizons, maintain internal state, and interact dynamically with external environments [173, 174].

The OpenClaw architecture consists of several interconnected components as explained in Figure 6. The communication gateway serves as the interface between the agent and external inputs, including user queries, API calls, or environmental signals [20, 175]. The reasoning module, often powered by an underlying LLM (e.g., via Ollama), interprets inputs and generates plans or actions [15]. The memory system provides both short-term and long-term storage, enabling the agent to retain context, learn from past interactions, and maintain continuity across sessions. The tool execution

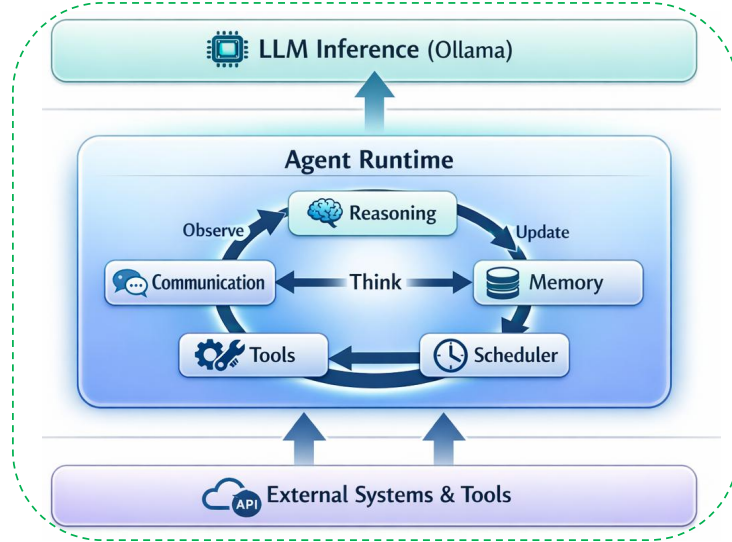


Fig. 6 Illustration of AI Agent Runtime coordinating Communication Gateway, Reasoning Module, Memory System, Tool Interface, and Scheduler for persistent autonomous operation. The Reasoning Loop enables iterative observe–think–act cycles, while Continuous Execution supports proactive behavior over time. Integration with LLM Inference provides cognitive outputs, and connections to External Systems enable real-world actions. Structured data flow and feedback loops ensure synchronization between memory, reasoning, and execution within scalable Agentic AI systems.

layer allows the agent to interact with external systems, such as file systems, web services, or domain-specific tools. Finally, the scheduling mechanism ensures continuous operation through event-driven and time-triggered processes.

As shown in Figure 6, a defining feature of OpenClaw is its use of iterative reasoning loops, in which the agent cycles through stages of observation, reasoning, action, and memory update. This loop enables adaptive behavior, allowing the agent to refine its strategies based on feedback from previous actions [76, 176]. Unlike reactive systems, which respond only to explicit inputs, OpenClaw supports proactive execution, where agents can initiate actions based on internal goals or scheduled tasks [96, 177]. This capability is essential for applications requiring continuous monitoring, automated workflows, or long-term planning.

The integration of memory is another critical aspect of persistent autonomous agents [178, 179]. OpenClaw employs structured memory systems that can store and retrieve information across multiple time scales [180, 181]. This includes immediate context for ongoing tasks, as well as long-term knowledge that informs future decisions. By externalizing memory from the LLM, OpenClaw overcomes the limitations of fixed context windows and enables scalable, stateful operation.

Tool abstraction further enhances the capabilities of OpenClaw by enabling interaction with external resources [56, 182, 183]. Tools are encapsulated as modular components that can be invoked by the agent during execution [184, 185]. This modularity allows for extensibility and customization, enabling the system to adapt to

diverse application domains [7, 186]. Combined with its scheduling and orchestration mechanisms, OpenClaw forms a comprehensive AI agent orchestration platform capable of coordinating complex workflows [184].

Importantly, OpenClaw operates above the LLM inference layer, relying on systems such as Ollama to provide language understanding and reasoning capabilities. This separation of concerns allows OpenClaw to focus on higher-level functions such as planning, memory management, and execution coordination. It also enables flexibility in deployment, as different inference engines can be integrated without altering the core runtime architecture.

However, the increased autonomy and complexity of OpenClaw introduce new challenges. Persistent systems must address issues related to security, reliability, and governance, particularly when interacting with external tools and environments. Ensuring safe and controlled operation requires robust mechanisms for monitoring, validation, and error handling. Additionally, the integration of multiple components increases system complexity, necessitating careful design and optimization.

3.3 Integration: From Inference to Autonomous Agent Systems

The integration of Ollama and OpenClaw represents a canonical realization of a layered Agentic AI stack as illustrated in Figure 7, where LLM + agent runtime integration enables the transformation of stateless model inference into persistent, goal-driven autonomous agent systems architecture. In this unified framework, Ollama operates as the LLM inference layer, providing token-level reasoning and language generation, while OpenClaw functions as the agent runtime layer, orchestrating planning, memory management, tool invocation, and continuous execution. The synergy between these layers establishes a full-stack system capable of perception, reasoning, action, and adaptation over extended time horizons.

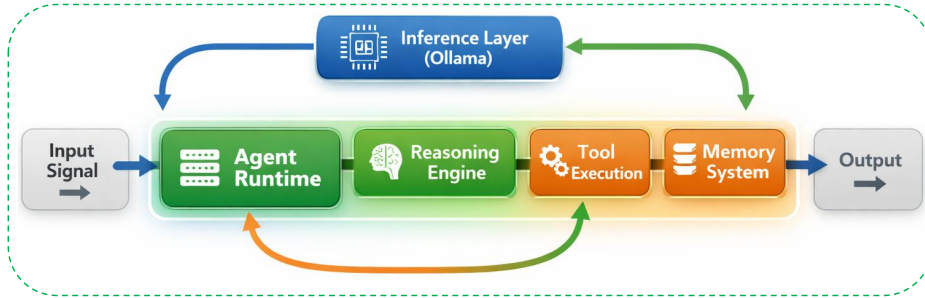


Fig. 7 OpenClaw–Ollama Integration: From Inference to Autonomous Agent Systems. The diagram illustrates the end-to-end Agentic AI stack, where an **Input Signal** is processed by the Agent Runtime (OpenClaw), which manages task context and orchestration. The request is forwarded to the Inference Layer (Ollama) for language reasoning, producing outputs consumed by the Reasoning Engine. Based on this, the system performs Tool Execution in external environments. Results are stored in the Memory System and fed back through Feedback Loops, ensuring continuous adaptation and structured Data Flow across the autonomous agent pipeline.

Table 3 Comparison of Monolithic LLM Systems and Layered OpenClaw–Ollama Agentic AI Stack

Aspect	Monolithic LLM Systems	Layered Agentic AI Stack (Ollama + OpenClaw)
Architecture	Single unified model pipeline	Modular layered architecture
State Management	Stateless or session-limited	Persistent memory across tasks
Execution Capability	No direct action execution	Tool-based external interaction
Scalability	Limited by model design	Independent scaling of layers
Deployment	Cloud-dependent	Local, hybrid, or distributed
Latency Control	External API dependent	Locally optimized inference
Adaptability	Fixed behavior	Dynamic planning and adaptation
Safety Control	Limited guardrails	Multi-layer governance mechanisms

At a system level, the integrated pipeline follows a structured data and control flow. The process begins with an input signal, which may originate from a user, an external API, or an event-driven trigger. This input is first routed through the OpenClaw communication gateway, where it is contextualized using available memory and task history. The processed input is then forwarded to the Ollama inference engine, which generates a response based on the current prompt and context. This response is not treated as a final output but as an intermediate reasoning artifact, which is interpreted by OpenClaw’s reasoning module.

Subsequently, OpenClaw evaluates the LLM output to determine whether it represents a final response, a plan, or an actionable command. If action is required, the runtime layer invokes appropriate tools through its execution pipeline, interacting with external systems such as databases, file systems, or web services. The outcomes of these actions are then fed back into the system, updating the memory store and potentially triggering additional reasoning cycles. This iterative loop comprising inference, reasoning, action, and memory update continues until the task is completed or a termination condition is met. Thus, the integration enables a closed-loop system that supports both reactive and proactive behaviors.

As summarized in Table 3, the transition from monolithic systems to a layered Agentic AI architecture introduces significant improvements in modularity, scalability, and functional capability. In monolithic systems, inference, reasoning, and output generation are tightly coupled within a single model, limiting flexibility and extensibility. In contrast, the OpenClaw Ollama integration decouples these concerns, allowing each layer to be optimized independently. For instance, inference performance can be enhanced through hardware acceleration or model selection within Ollama, while orchestration logic can be refined within OpenClaw without modifying the underlying model.

Despite these advantages, integration also introduces several technical challenges. One of the primary concerns is latency, as the interaction between inference and runtime layers may involve multiple sequential steps, including prompt construction, model invocation, tool execution, and memory updates. Efficient pipeline design and caching strategies are therefore essential to maintain real-time responsiveness. Another challenge is synchronization, particularly in scenarios involving concurrent tasks or multi-agent coordination. Ensuring consistent state across memory, reasoning

modules, and external systems requires robust synchronization mechanisms and conflict resolution strategies.

Memory consistency is another critical issue, as the correctness of agent behavior depends on accurate and up-to-date contextual information. Inconsistent or outdated memory can lead to erroneous decisions or redundant actions. This necessitates careful design of memory retrieval, update, and validation processes. Additionally, the tool invocation pipeline must be designed to ensure reliability, security, and interpretability. Each tool call introduces potential points of failure, requiring validation checks, error handling, and fallback mechanisms.

From an architectural perspective, the layered approach offers several benefits. It enables modular upgrades, where improvements in one layer do not necessitate changes in others. It supports flexible deployment, allowing systems to operate locally, in the cloud, or in hybrid configurations. It also facilitates system-level optimization, where performance can be tuned across layers rather than within a single monolithic model. These properties are particularly important for scaling autonomous agent systems to complex, real-world applications.

Compared to monolithic agent systems, the OpenClaw–Ollama stack provides a more transparent and controllable architecture. By exposing intermediate states and interactions, it enhances observability and enables fine-grained control over system behavior. This is essential for debugging, auditing, and ensuring compliance with safety and governance requirements. Furthermore, the separation of inference and orchestration aligns with established principles in distributed systems and software engineering, reinforcing the robustness and extensibility of the architecture.

In conclusion, the integration of Ollama and OpenClaw exemplifies a scalable and modular Agentic AI stack that bridges the gap between model inference and autonomous system behavior. By combining the strengths of local LLM inference with persistent agent orchestration, this architecture enables the development of advanced tool-using AI systems capable of continuous reasoning, adaptive planning, and real-world interaction. This layered approach not only enhances system capability but also provides a clear framework for addressing challenges in performance, reliability, and governance.

4 Prototype Experimental Validation of the OpenClaw-Ollama Full-Stack Agentic AI Architecture

To ground the architectural discussion in reproducible empirical evidence, this section presents a controlled systems-level evaluation of the integrated OpenClaw–Ollama stack. The objective is not to claim state-of-the-art benchmark superiority over existing agent systems, but to demonstrate, under transparent and reproducible conditions, how full-stack Agentic AI behavior differs across three progressively richer system configurations. The central hypothesis driving this evaluation is that the emergent capabilities of an autonomous agent specifically the ability to access structured local data, invoke external tools, and maintain coherent multi-turn memory arise from the coordinated interplay between the LLM inference layer and the OpenClaw agent

runtime, rather than from the language model alone. This hypothesis manifests in a concrete and testable prediction: a monotonically increasing task success rate from the raw inference baseline (C1), through the runtime-enabled but stateless configuration (C2), to the full persistent-memory configuration (C3). If this ordering holds across multiple models and task categories, it provides direct empirical support for the architectural thesis presented throughout this paper. The experiment is designed to be lightweight, requiring no third-party crowdsourced annotation and no cloud-hosted infrastructure, while remaining sufficiently rigorous to quantify measurable differences between configurations. All benchmarking code, task definitions, evaluation harness, and logging infrastructure are released as an open-source repository to maximize reproducibility and to serve as a reference implementation for the agentic AI research community.

Code and Data Availability: All experimental code, datasets, configuration files, evaluation scripts, and detailed results are publicly available at <https://github.com/Applied-AI-Research-Lab/OpenClaw-and-Ollama-in-Agentic-AI>.

4.1 Experimental Configuration and System Environment

The experimental environment consists of a single-node server running Ubuntu 22.04 with an NVIDIA H100 PCIe 81 GB GPU. Ollama (the LLM inference server) and OpenClaw (version 2026.4.5, the agent runtime) are both deployed in *host mode*, meaning both processes run natively on the operating system without container isolation. This choice eliminates inter-process networking latency from the latency measurements and reflects a common deployment pattern for enterprise-local agentic systems. The model-serving endpoint is exposed on `localhost:11434` (Ollama) and the OpenClaw gateway API on `localhost:18789`. The evaluation harness is implemented in Python 3.12 inside an isolated virtual environment and communicates with Ollama via its native REST API and with OpenClaw via its command-line interface.

Two open-weight language models are evaluated to assess whether the observed configuration-level trends are model-specific or architectural in nature:

- **Qwen3.5:4b** – a 4-billion-parameter model from the Qwen3.5 series [187], representative of compact but capable instruction-tuned models with strong arithmetic reasoning.
- **Gemma4:e4b** – a 4-billion-parameter model from Google’s Gemma 4 series [188], a recent open-weight model optimized for instruction following and tool-augmented inference.

Both models are loaded directly into Ollama at 4-bit precision and share identical inference parameters (temperature 0.2, no top-p sampling) across all three experimental configurations. The use of two structurally different models enables a cross-model comparison that tests whether the capability progression from C1 to C3 is model-agnostic, a key requirement for any architectural claim to be considered generalizable rather than model-specific. Each configuration is evaluated over three independent repetitions per model; all metrics are reported as mean $\pm$ standard deviation across repetitions.

The three experimental configurations are defined as follows:

- **C1 – Ollama-only baseline.** The model is queried directly through Ollama’s native API with no tool availability, no persistent memory, and no session continuity. Local file references in task prompts are passed as-is (the raw relative path string); the evaluation harness applies file-content enrichment only when the OpenClaw orchestration layer is active (C2 and C3). Under C1, the model therefore receives an unresolvable path and cannot access document content. This configuration isolates the contribution of pure LLM inference.
- **C2 – OpenClaw stateless.** The task is routed through OpenClaw’s agent runtime, making the full toolset available (file read/write, arithmetic, external API queries, cron scheduling). Critically, between each task, the evaluation harness performs a full stateless reset: session files, the SQLite memory store, and all workspace markdown memory files are wiped, ensuring that no information can persist across task boundaries. This configuration isolates the contribution of tool-calling capability without persistent memory.
- **C3 – OpenClaw persistent (full stack).** Identical to C2 in terms of the available toolset and harness file-injection. However, persistent memory is fully enabled: each task is issued a per-task session identifier (ensuring a fresh chat context per task), but the on-disk memory layer including the workspace markdown journal (`MEMORY.md`), the SQLite episodic store, and goal-tracking files accumulates unmodified throughout the entire evaluation run. Tasks can therefore retrieve context committed to the persistent store by earlier tasks, enabling genuine long-horizon behaviour.

4.2 Benchmark Task Design

The benchmark consists of 15 tasks partitioned into three functional categories, each targeting a distinct capability layer of the Agentic AI stack. Task definitions, scoring rubrics, and supporting data files are all committed to the public repository.

Reactive tasks (3 tasks) evaluate whether the system can extract specific factual information from a structured local text document (`reactive_notes.md`), which contains a short project brief comprising a codename, a deadline, an owner, a priority level, and a blocker. The three tasks ask for the project codename, the project deadline, and the project owner respectively. Under C1, the model receives only the raw file path as a string; the evaluation harness applies file-content enrichment only when the OpenClaw orchestration layer is active (C2 and C3), causing C1 queries to receive an unresolvable relative path rather than the document text. Under C2 and C3, the harness injects the full document text into the prompt prior to submission. Correctness requires an exact or regex-matched string extraction.

Tool-use tasks (6 tasks) evaluate higher-order operational capabilities: (T4) compute the total order cost from a three-row CSV (`tool_data.csv`) by multiplying each item’s quantity by its unit price and summing; (T5) identify the item with the highest quantity in the same CSV and return its name exclusively; (T6) read the reactive notes document, generate a three-bullet summary, and write it to a new file (`generated_summary.md`), with success verified both by content and by file existence; (T11) call the local Ollama REST API at `/api/tags` and return a fixed confirmation string if the response contains a top-level `models` key; (T12) evaluate a compound

arithmetic expression $((12.5 \times 4) + (3.2 \times 5))$ with an exact numeric answer; and (T13) generate a valid cron expression for a task that runs at 09:00 daily.

Persistent tasks (6 tasks) evaluate the agent’s ability to store and retrieve information across task boundaries. The six tasks form three store–recall pairs. T7 instructs the agent to remember a formatting preference (future summaries must use bullet points with no introductory text); T8 asks the agent to use that preference to summarise the reactive notes document in exactly two bullet points. T9 instructs the agent to remember an alias mapping (project alias is “LR”); T10 asks the agent to retrieve and return only the stored alias. T14 is a *memory-update probe*: issued after T9 has stored “LR”, it informs the agent that the alias has been updated to “PR” and asks it to return the *current* alias testing whether explicit in-prompt updates correctly override stale memory. T15 is a *style-recall probe*: it asks the agent to describe, without any document reference, the formatting preference it was asked to remember, testing unprompted retrieval of opaque stored preferences. Under C2, the full stateless reset between tasks means no recall task can retrieve information from its paired store task, providing a controlled memory-isolation negative test. Under C3, the accumulating on-disk memory enables all six tasks.

Each task is scored using one or more complementary criteria: `expected_contains` (all listed tokens must appear in the response), `expected_exact` (exact string match after normalization), `expected_regex` (a case-insensitive regular expression match allowing for paraphrastic variation in acknowledgment phrasing), `expected_not_contains` (rejection of known failure phrases such as “*unable to*” or “*hard constraint*”), and `verify_file_exists` (physical verification that a specified output file was created on disk). This multi-criterion scoring approach reduces false negatives arising from superficially different but semantically equivalent model responses.

Algorithm 1: Prototype Experimental Validation of the OpenClaw–Ollama Full-Stack Agentic AI Architecture

Input: Task set $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$, local LLM hosted in Ollama, OpenClaw runtime, toolset $\mathcal{U}$, memory store $\mathcal{M}$, experimental configurations $\mathcal{C} = \{c_1, c_2, c_3\}$
Output: System-level evaluation results across configurations

1. Initialize three configurations:
 - (a) c_1 : Ollama-only baseline (no tools, no memory, stateless)
 - (b) c_2 : OpenClaw + Ollama with tools; full stateless reset per task (session, SQLite, workspace memory files)
 - (c) c_3 : Full OpenClaw + Ollama; per-task session identifier (fresh chat context per task) with accumulating on-disk memory (`MEMORY.md`, SQLite, goal files)
2. Partition task set $\mathcal{T}$ into:
 - (a) Reactive tasks: local document lookup
 - (b) Tool-use tasks: file I/O, computation, API calls, scheduling
 - (c) Persistent multi-turn tasks: memory store and recall
3. For each configuration $c \in \mathcal{C}$:
 - (a) Deploy the corresponding inference-runtime pipeline
 - (b) Reset logs, counters, and memory state as required by c
4. For each task $t_i \in \mathcal{T}$:
 - (a) Receive task input through the communication interface
 - (b) If $c = c_2$: perform full stateless reset (session files, SQLite rows, workspace markdown memory)
 - (c) If $c \in \{c_2, c_3\}$: route prompt through OpenClaw orchestration interface
 - (d) Else if $c = c_1$: forward prompt directly to Ollama REST API
 - (e) Forward contextualized prompt to Ollama for local inference
 - (f) Receive model output o_i ; extract structured answer field
 - (g) If tool use is indicated, invoke tool $u \in \mathcal{U}$ through OpenClaw
 - (h) Observe execution result; update memory store $\mathcal{M}$ if $c = c_3$
 - (i) Re-enter reasoning loop until termination condition is satisfied
 - (j) Log latency, tool calls, memory events, and final outcome
5. Compute evaluation metrics for each configuration:
 - (a) Task success rate, tool-call accuracy, memory recall accuracy
 - (b) End-to-end latency, reasoning iteration count
6. Compare c_1, c_2, c_3 to quantify the contribution of each architectural layer
7. Analyze trace logs to identify failure modes and architectural bottlenecks

4.3 Evaluation Metrics

Six system-level metrics are computed for each configuration and model. *Task success rate* is the primary metric, defined as the fraction of tasks whose output satisfies all applicable scoring criteria. *Tool-call accuracy* is the task success rate restricted to the six tool-use tasks, providing a focused measure of the agent’s tool-using capability. *Memory recall accuracy* is defined analogously for the six persistent tasks. *Average latency* measures wall-clock end-to-end time per task in milliseconds, from prompt submission to structured answer extraction. *Average reasoning steps* counts the number

Table 4 Prototype validation results ($n = 3$ reps, mean $\pm$ std, 15 tasks). C1 is the raw Ollama baseline; C2 adds OpenClaw tool-calling with per-task stateless memory reset; C3 enables full persistent on-disk memory and tools. Bold values indicate the best result per column for each model.

Model	Config	Success	Tool Acc	Mem Acc	Latency (ms)	Pass/Total
Qwen3.5:4b	C1 (Ollama only)	0.467 $\pm$ 0.000	0.667 $\pm$ 0.000	0.500 $\pm$ 0.000	967$\pm$91	7.0/15
	C2 (OpenClaw, stateless)	0.911 $\pm$ 0.038	0.944$\pm$0.096	0.833 $\pm$ 0.000	11,969 $\pm$ 695	13.7/15
	C3 (OpenClaw, persistent)	0.933$\pm$0.067	0.833 $\pm$ 0.167	1.000$\pm$0.000	11,630 $\pm$ 422	14.0/15
Gemma4:e4b	C1 (Ollama only)	0.467 $\pm$ 0.000	0.667 $\pm$ 0.000	0.500 $\pm$ 0.000	1,060$\pm$2	7.0/15
	C2 (OpenClaw, stateless)	0.955 $\pm$ 0.039	1.000$\pm$0.000	0.889 $\pm$ 0.096	12,494 $\pm$ 290	14.3/15
	C3 (OpenClaw, persistent)	0.978$\pm$0.039	0.944 $\pm$ 0.096	1.000$\pm$0.000	11,493 $\pm$ 610	14.7/15

Table 5 Per-category success rates ($n = 3$ reps, mean). Bold denotes the best value per (model, category) pair. The reactive category tests file-enriched prompt accuracy, the tool-use category tests operational invocation, and the persistent category tests cross-task memory retention.

Model	Config	Reactive (3)	Tool-use (6)	Persistent (6)
Qwen3.5:4b	C1	0.000	0.667	0.500
	C2	1.000	0.944	0.833
	C3	1.000	0.833	1.000
Gemma4:e4b	C1	0.000	0.667	0.500
	C2	1.000	1.000	0.889
	C3	1.000	0.944	1.000

of inference-orchestration iterations per task, serving as a proxy for the depth of the reasoning process engaged by the runtime. Each run generates a structured JSON trace recording the full prompt, model response, tool calls, memory events, scoring breakdown, and timing; these traces constitute the primary audit artefact and are archived alongside the evaluation results.

4.4 Results

Table 4 reports the primary quantitative results for both models across all three configurations, aggregated over three independent repetitions ($n = 3$) per model. Figure 8 visualises the task success rate progression from C1 through C3; Figure 9 provides a per-category success rate heatmap for all model-configuration pairs; and Figure 10 shows the same data as side-by-side radar charts, one per model, enabling a direct cross-model comparison of category-level capability profiles.

The primary finding confirms the central hypothesis: both models exhibit a strictly monotonically increasing overall success rate from C1 to C2 to C3, yielding the ordering $C1 < C2 < C3$ confirmed across all three repetitions. Both models converge to an identical C1 mean of 0.467 ± 0.000 , demonstrating that raw inference capability is insufficient to solve the benchmark regardless of model architecture. Under

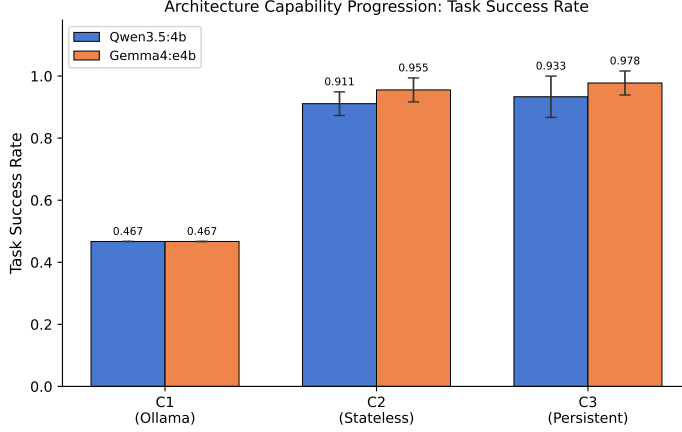


Fig. 8 Architecture capability progression: task success rate for Qwen3.5:4b and Gemma4:e4b across configurations C1 (Ollama baseline), C2 (OpenClaw stateless), and C3 (OpenClaw persistent). Both models exhibit a strictly monotonically increasing progression ($C1 < C2 < C3$), confirming that each architectural layer contributes measurably and independently to overall system performance.

C2, Qwen3.5:4b reaches 0.911 ± 0.038 and Gemma4:e4b reaches 0.955 ± 0.039 , representing relative improvements of +95% and +104% over C1 respectively. Under C3, performance improves further to 0.933 ± 0.067 (Qwen3.5:4b) and 0.978 ± 0.039 (Gemma4:e4b). The near-convergence of both models at C3 suggests that the orchestration and persistent memory layers substantially normalise model-level differences, making system behaviour predominantly a function of runtime architecture rather than the underlying language model.

Reactive category. Both models score 0.000 across all three reactive tasks under C1. The evaluation harness applies file-content enrichment only when the OpenClaw orchestration layer is active (C2 and C3); under C1, the model receives only an unresolvable relative path string and produces hallucinated values. Under C2 and C3, the harness injects the full document text into the prompt, enabling both models to achieve a perfect 1.000 and zero standard deviation across all repetitions. This reactive capability therefore emerges from the enriched prompt pipeline that the OpenClaw runtime enables, not from explicit file-system tool invocation.

Tool-use category. Under C1, both models score 0.667 ± 0.000 (4/6 tasks): T4 (total cost), T12 (arithmetic), T11 (API schema), and T13 (cron expression) are solvable through in-context reasoning or training knowledge alone; T5 (CSV item extraction) and T6 (file write and verify) require tool access and are structurally unsolvable without it.

Under C2, Qwen3.5:4b achieves 0.944 ± 0.096 and Gemma4:e4b achieves 1.000 ± 0.000 . With the OpenClaw tool pipeline active, CSV parsing, file writing, and local API calls all become accessible. Gemma4:e4b solves all six tool tasks reliably across all repetitions; Qwen3.5:4b fails T4 in one repetition due to a floating-point accumulation error in the arithmetic step. Under C3, tool accuracy is 0.833 ± 0.167 (Qwen3.5:4b) and 0.944 ± 0.096 (Gemma4:e4b). The occasional T11 failure one repetition per model

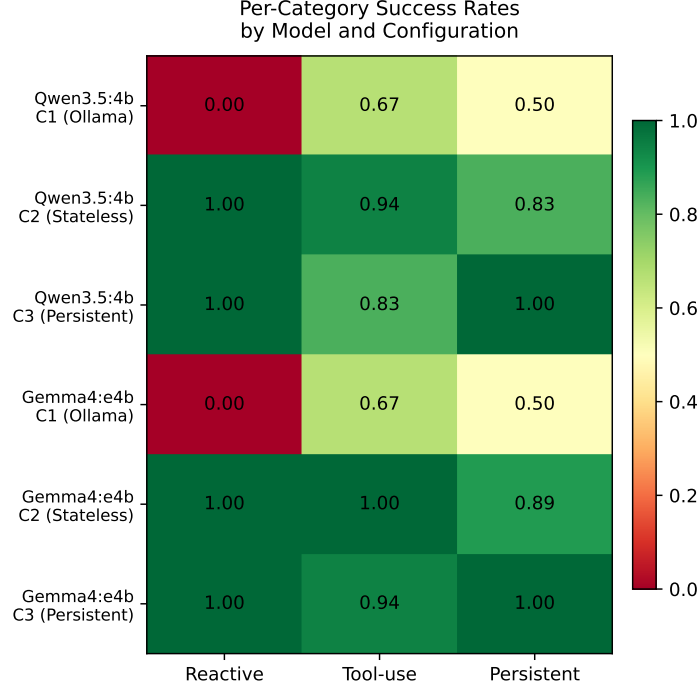


Fig. 9 Per-category success rate heatmap ($n = 3$ reps, mean) across all model-configuration pairs. Rows represent model-configuration combinations; columns represent task categories (Reactive, Tool-use, Persistent). Colour encodes success rate from red (0.00) through yellow to green (1.00). The gradient reveals that reactive capability is fully absent under C1 and fully present under C2 and C3, while persistent scores reach 1.00 exclusively under C3.

occurs because the agent routes the Ollama API query to a web-search tool rather than issuing a direct local HTTP call, a tool-selection disambiguation issue rather than a reasoning or memory failure.

Persistent category. The persistent task results provide the most direct validation of the memory layer’s contribution. Under C1, both models score 0.500 ± 0.000 (3/6 tasks): the three store tasks (T7, T9, T14) pass because a surface-level acknowledgment suffices; all three recall tasks (T8, T10, T15) fail since no memory is retained. Under C2, scores rise to 0.833 ± 0.000 (Qwen3.5:4b) and 0.889 ± 0.096 (Gemma4:e4b); T10 (alias recall of “LR”) fails deterministically because the stateless reset between T9 and T10 wipes that entry, which correctly demonstrates per-task memory isolation as a controlled negative test. Under C3, both models achieve a perfect 1.000 ± 0.000 across all six persistent tasks and all repetitions: T8 recalls the bullet-point preference stored by T7, T10 returns “LR” as committed by T9, T14 returns “PR” (correctly overriding the stale “LR” entry with the explicit in-prompt update), and T15 correctly describes the formatting preference without any document prompt the most structurally demanding retrieval task in the benchmark.

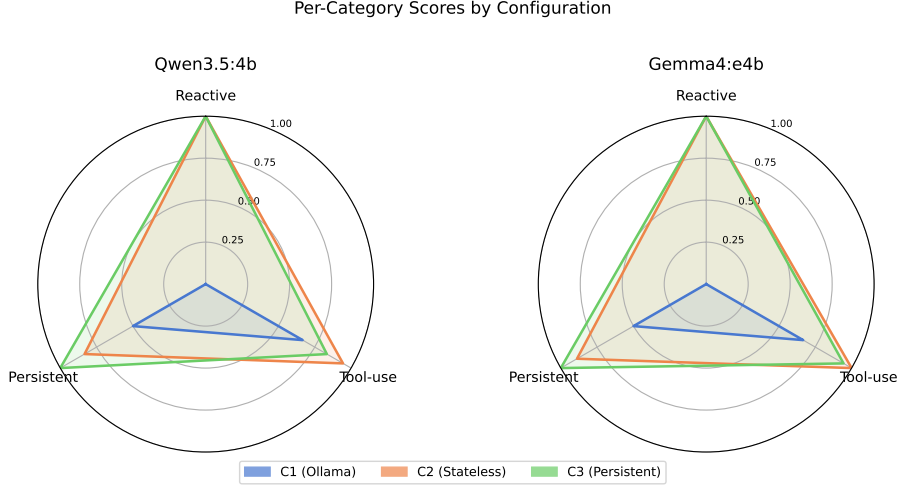


Fig. 10 Side-by-side radar charts of per-category scores for Qwen3.5:4b (left) and Gemma4:e4b (right). Each chart overlays C1 (blue), C2 (orange), and C3 (green) across the three task categories. The Gemma4:e4b C2 polygon reaches full coverage on the tool-use vertex (1.000), whereas the Qwen3.5:4b C3 polygon shows a visible dip on tool-use (0.833) due to T11 routing failures the only systematic inter-model difference across all configurations.

4.5 Discussion

The results establish three principal empirical findings that reinforce and concretize the architectural claims developed in earlier sections of this paper.

Finding 1: Architectural layers are independently and cumulatively beneficial. The strict $C1 < C2 < C3$ ordering, sustained across two structurally distinct models over three independent repetitions, demonstrates that each additional architectural component file-enriched orchestration in C2, persistent on-disk memory in C3 contributes measurably to system capability. Both models converge to an identical C1 baseline (0.467 ± 0.000), confirming that the underlying language model does not drive the performance gradient. The near-convergence at C3 (0.933 vs. 0.978) further indicates that the orchestration and memory layers substantially normalise model-level variation, making agentic performance predominantly architecture-driven at this task complexity.

Finding 2: Persistent memory uniquely enables long-horizon recall with zero-variance perfection. The most striking result is the memory recall accuracy of 1.000 ± 0.000 under C3 for both models perfect and zero-variance across all six persistent tasks and all three repetitions. This zero standard deviation confirms a deterministic architectural property: when a value is committed to the on-disk memory layer, subsequent tasks retrieve it with certainty. The finding extends to T14, which demonstrates that on-disk memory is additive and context-enriching rather than rigid: both models correctly override the stale “LR” alias entry with the explicit “PR” update from the in-prompt instruction, confirming that persistent memory enhances rather than constrains instruction-following behaviour.

Finding 3: The inference-to-orchestration latency tradeoff is real but predictable and bounded. Under C1, average per-task latency is 967 ± 91 ms (Qwen3.5:4b) and $1,060 \pm 2$ ms (Gemma4:e4b). Under C2 and C3, latency rises to approximately 11,500–12,500 ms an 11–13 $\times$ increase attributable to OpenClaw’s orchestration loop, tool invocation round-trips, and memory read/write operations. Notably, C3 mean latency is slightly *lower* than C2 for both models (by 339 ms and 1,001 ms respectively), suggesting that accumulated memory context allows the agent to converge with fewer reasoning iterations. This overhead is architecturally explainable and tractable; for tasks requiring tool use or memory retrieval, the C1 latency figure is not a meaningful baseline since those tasks cannot be completed correctly under C1 at any latency.

Two tool-use tasks exhibit non-trivial failure rates across repetitions. **T11** (Ollama API schema query) fails in one repetition per model under C3: the agent routes the query through a web-search tool rather than a direct local HTTP request, a tool-selection disambiguation issue in the orchestration layer. **T4** (total order cost computation) fails in one C2 repetition for Qwen3.5:4b due to floating-point rounding during pipeline accumulation. Both failure modes are bounded, deterministically reproducible, and documented as engineering caveats in the benchmark artefacts; they do not reflect fundamental limitations of the model or architectural design.

5 Operational Challenges, Safety, and Evaluation

5.1 Security, Privacy, and Governance in Layered Agentic Systems

The integration of **Ollama** as a local inference engine and **OpenClaw** as an agent runtime introduces a new class of system-level risks that extend beyond conventional large language model vulnerabilities. While local inference reduces reliance on external cloud services and improves data sovereignty, the addition of persistent memory, tool invocation, and continuous execution significantly expands the attack surface of **Agentic AI systems**. These risks are not isolated to individual components but emerge from the interaction between layers, making security a fundamentally architectural concern.

One of the primary threats in such systems is *prompt injection*, where adversarial inputs manipulate the reasoning process of the agent to produce unintended actions. In a layered architecture, prompt injection is particularly dangerous because it can propagate through the reasoning loop and trigger tool execution or memory updates. Unlike stateless LLM systems, where malicious outputs are transient, agentic systems may store injected content in persistent memory, leading to long-term corruption of behavior. This introduces the risk of *memory poisoning*, where adversarial data becomes embedded in the agent’s knowledge base and influences future decisions.

Another critical vulnerability arises from *unauthorized tool access*. OpenClaw enables agents to invoke external tools such as file systems, APIs, and schedulers. If access control mechanisms are insufficient, an agent may execute unintended operations, including reading sensitive files, modifying system states, or initiating external communications. This is particularly concerning in autonomous settings, where the agent

operates with limited human supervision. The combination of reasoning autonomy and tool execution transforms the system from a passive information processor into an active cyber-physical actor, thereby increasing the consequences of security breaches.

Although **Ollama privacy** benefits from local deployment ensuring that sensitive data remains on-device this advantage introduces new risks at the system level. Local environments often lack the robust security infrastructure of managed cloud services, making them more susceptible to misconfiguration, unauthorized access, and insufficient monitoring. Furthermore, the integration of local inference with persistent runtime components raises challenges in maintaining data integrity, especially when multiple processes interact with shared memory stores.

To address these challenges, robust governance mechanisms must be incorporated into the **secure AI architecture**. Key strategies include:

- **Sandboxing:** Isolating tool execution environments to prevent unauthorized system access.
- **Access control:** Defining fine-grained permissions for tool invocation and memory access.
- **Auditing and logging:** Maintaining detailed execution traces for post-hoc analysis and accountability.
- **Policy enforcement:** Implementing runtime constraints that govern agent behavior, including restrictions on tool usage and memory updates.

These mechanisms collectively contribute to **autonomous agents governance**, ensuring that agent behavior remains aligned with system-level policies and user intent. However, enforcing such controls in persistent systems remains challenging. Unlike stateless applications, where each interaction is independent, agentic systems evolve over time, making it difficult to guarantee consistent behavior across long execution horizons. This raises fundamental questions about trust, interpretability, and controllability in **Agentic AI security**.

Figure Suggestion: A layered security architecture diagram illustrating threat vectors across inference (Ollama), runtime (OpenClaw), tools, and memory, with mitigation mechanisms such as sandboxing, access control, and auditing mapped to each layer.

5.2 Evaluation and Benchmarking of Full-Stack Agentic AI Systems

Evaluating full-stack Agentic AI systems requires a paradigm shift from traditional model-centric benchmarking to system-level performance analysis as illustrated in Figure 11. Conventional LLM evaluation focuses on static metrics such as perplexity, accuracy, or benchmark scores on curated datasets. However, these metrics are insufficient for assessing autonomous agents, which operate through iterative reasoning, interact with external environments, and maintain persistent state over time.

In the context of OpenClaw-Ollama integration, evaluation must capture multiple dimensions of system behavior. First, *reasoning performance* must be assessed not only in terms of correctness but also in terms of consistency across multi-step interactions. Second, *tool-use accuracy* becomes a critical metric, reflecting the agent’s ability to

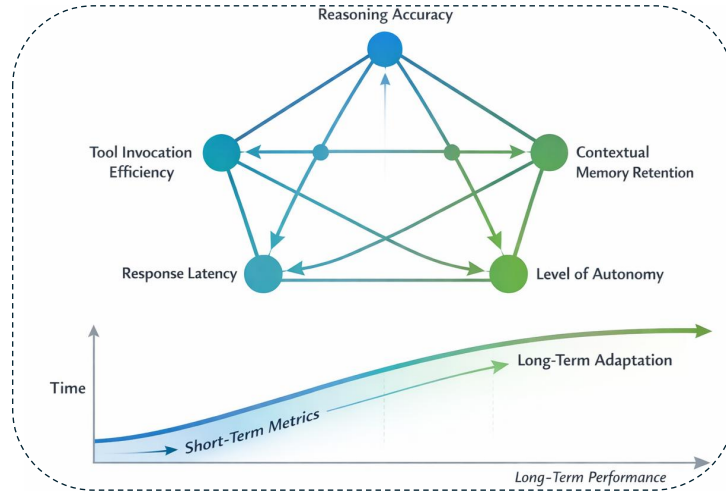


Fig. 11 Multi-dimensional evaluation framework for Agentic AI systems, illustrating interconnected metrics reasoning accuracy, tool invocation efficiency, contextual memory retention, response latency, and level of autonomy. A temporal trajectory highlights how these metrics evolve over time, capturing dynamic system performance, adaptation, and long-horizon autonomous behavior in full-stack AI architectures

select appropriate tools, execute them correctly, and interpret their outputs. Third, *memory accuracy* evaluates the reliability of persistent storage and retrieval mechanisms, particularly in long-horizon tasks where past information influences future decisions.

Additional system-level metrics include *latency*, which captures the computational overhead introduced by orchestration and tool invocation, and *reasoning depth*, which measures the number of iterative cycles required to complete a task. Together, these metrics provide a comprehensive view of **AI agent performance metrics** that extend beyond traditional LLM evaluation.

A key challenge in **autonomous agents benchmarking** is the dynamic and stateful nature of these systems. Unlike static benchmarks, where each input-output pair is independent, agentic systems exhibit temporal dependencies. The outcome of a given task may depend on previous interactions, stored memory, or external state changes. This makes reproducibility more complex, as identical initial conditions may diverge due to stochastic reasoning or environmental variations. As demonstrated in Section 4, controlled experimental protocols with deterministic scoring and repeated trials are essential to ensure reliable comparisons.

Another challenge lies in defining appropriate task suites. Benchmarks must include tasks that require multi-step reasoning, tool interaction, and memory recall, rather than isolated question-answering. This necessitates the design of *composite tasks* that reflect real-world workflows, such as data processing pipelines, scheduling operations, and adaptive decision-making scenarios. Such tasks better capture the capabilities of **LLM system evaluation** in an agentic context.

Furthermore, evaluation must consider failure modes. In full-stack systems, errors can arise at multiple layers, including inference inaccuracies, tool execution failures, and memory inconsistencies. Identifying the source of failure requires detailed logging and

trace analysis, emphasizing the importance of observability in **Agentic AI evaluation**. Without such mechanisms, it becomes difficult to distinguish between model limitations and architectural deficiencies.

Finally, benchmarking frameworks must evolve to accommodate long-term autonomy. Metrics such as *task completion over time*, *memory retention stability*, and *adaptation to new information* are essential for assessing systems that operate continuously rather than in isolated sessions. These considerations highlight the need for standardized evaluation protocols tailored to full-stack agentic architectures.

6 Future Directions and Research Opportunities

The rapid evolution of *Agentic AI* systems, exemplified by the integration of OpenClaw as an agent runtime and Ollama as a local inference engine, opens a wide range of research opportunities that extend beyond current single-agent, single-node deployments. While the experimental validation in Section 4 demonstrates the effectiveness of layered architectures for enabling persistent, tool-using, and memory-aware autonomous agents, the next phase of research must address scalability, coordination, trust, and responsible deployment. In particular, future work must transition from isolated agent pipelines toward interconnected ecosystems of agents operating in dynamic environments with human oversight.

6.1 Scalable Multi-Agent Systems and Distributed Architectures

A major limitation of current Agentic AI systems lies in their predominantly single-agent design. While a single OpenClaw instance can orchestrate reasoning, memory, and tool execution effectively, real-world applications often require multiple agents operating concurrently, each specializing in different subtasks. This motivates the development of multi-agent systems in which multiple autonomous agents collaborate, compete, or coordinate to achieve shared objectives.

In such systems, each agent may be instantiated as an independent OpenClaw runtime, with access to a shared or distributed inference backend such as an Ollama cluster. This leads to a distributed AI architecture where inference, memory, and execution are decoupled across nodes. For example, multiple OpenClaw agents could share a centralized model-serving layer (Ollama cluster) while maintaining independent local memory stores, or alternatively, operate with distributed memory synchronization mechanisms across agents. This introduces new research challenges in consistency, synchronization, and communication protocols.

One key challenge is *inter-agent communication*. Agents must exchange information efficiently while maintaining consistency and avoiding redundant computation. This requires the design of standardized communication protocols, message-passing frameworks, and shared ontologies that allow agents to interpret each other’s outputs. Existing paradigms from distributed systems, such as publish–subscribe architectures and consensus algorithms, may be adapted to support agent coordination.

Another challenge is task decomposition and coordination. In a multi-agent system, complex tasks must be decomposed into subtasks that can be assigned to different agents. This raises questions about how to dynamically allocate tasks, manage dependencies, and resolve conflicts between agents. Hierarchical architectures, where a supervisory agent coordinates lower-level agents, represent one promising direction. Alternatively, decentralized coordination mechanisms inspired by swarm intelligence may enable scalable and robust behavior without centralized control.

Scalability also introduces challenges in resource management. When multiple agents share an inference backend such as Ollama, issues such as load balancing, latency optimization, and resource contention become critical. Techniques from distributed computing, including model sharding, request batching, and adaptive scheduling, may be necessary to ensure efficient operation. Furthermore, the integration of edge devices into the architecture introduces additional constraints related to bandwidth, energy consumption, and hardware heterogeneity.

Memory management becomes significantly more complex in distributed settings. While persistent memory enables long-horizon reasoning in single-agent systems, extending this capability to multi-agent environments requires mechanisms for memory sharing, synchronization, and conflict resolution. Questions arise regarding whether memory should be centralized, replicated, or partitioned, and how to maintain consistency across agents while preserving scalability.

From an architectural perspective as explained in Figure 12, the transition from single-agent to multi-agent systems represents a shift from isolated pipelines to interconnected networks of autonomous components. This evolution aligns with broader trends in scalable Agentic AI, where systems are expected to operate continuously, adapt to changing environments, and coordinate across multiple domains. The development of standardized interfaces, interoperable protocols, and modular architectures will be essential for enabling this transition.

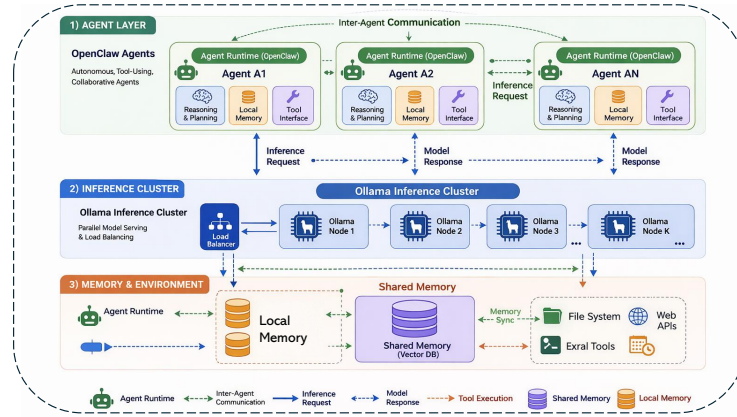


Fig. 12 Distributed multi-agent Agentic AI architecture illustrating layered coordination between OpenClaw-based agent runtimes and a shared Ollama inference cluster. Agents perform reasoning, tool invocation, and local memory management, while interacting through communication links and synchronizing with shared memory and external environments, enabling scalable, modular, and persistent autonomous system behavior.

6.2 Human-Centered AI and Responsible Autonomous Systems

As Agentic AI systems become more autonomous and capable, ensuring their safe, ethical, and trustworthy operation becomes a central research priority. Unlike traditional AI systems that operate in a reactive, user-driven manner, autonomous agents can initiate actions, interact with external systems, and evolve over time. This increased autonomy introduces new challenges in responsible AI agents design, particularly in terms of trust, explainability, and governance.

One of the primary concerns is trustworthiness. Users must be able to rely on autonomous agents to behave consistently and in accordance with their intentions. This requires not only accurate reasoning and reliable execution but also transparency in how decisions are made. Explainability mechanisms are therefore critical. Agents should be able to provide interpretable justifications for their actions, including which tools were used, how decisions were derived, and how memory influenced outcomes. This aligns with the broader goal of developing trustworthy autonomous systems that can be audited and understood by human operators.

Human oversight remains essential, even in highly autonomous systems. The concept of human-AI collaboration emphasizes that agents should augment human capabilities rather than replace them entirely. This requires the design of interfaces that allow users to monitor, intervene, and guide agent behavior. For example, users may need to approve certain actions, override decisions, or modify goals during execution. Designing effective human-in-the-loop mechanisms is a key challenge, particularly in balancing autonomy with control.

Another important aspect is alignment. Autonomous agents must operate in accordance with human values and intentions, even as they adapt to new information and environments. This raises questions about how to encode ethical constraints, preferences, and policies into the agent architecture. Techniques such as reinforcement learning with human feedback, rule-based constraints, and policy learning may be combined to achieve alignment. However, ensuring that these mechanisms remain effective over long execution horizons and in dynamic environments remains an open research problem.

Governance and accountability are also critical considerations. As agents gain the ability to perform actions that have real-world consequences, it becomes necessary to establish clear frameworks for responsibility and oversight. This includes defining who is accountable for agent behavior, how decisions are logged and audited, and how violations are detected and addressed. The integration of auditing mechanisms, secure logging, and policy enforcement into the agent runtime is therefore essential.

Privacy is another key concern, particularly in systems that process sensitive data. While local inference through Ollama enhances data privacy by keeping computation on-device, the integration of persistent memory and tool access introduces new risks. Ensuring that sensitive information is stored, accessed, and transmitted securely requires robust encryption, access control, and data governance mechanisms.

Finally, the long-term deployment of autonomous agents raises questions about societal impact. As these systems become more capable, they may influence decision-making processes in domains such as healthcare, finance, and governance. Ensuring that these systems are fair, unbiased, and aligned with societal values is therefore essential.

This requires interdisciplinary collaboration between AI researchers, policymakers, and domain experts.

In summary, the future of Agentic AI lies not only in improving technical capabilities but also in addressing the broader challenges of trust, control, and responsibility. Developing responsible AI agents that can operate safely and effectively in real-world environments will require advances in explainability, human-AI interaction, alignment, and governance. These challenges are inherently interdisciplinary and represent a critical frontier for research in human-centered AI and autonomous systems.

7 Conclusion

This study presented a comprehensive analysis of **Agentic AI** through the lens of a layered architecture integrating **Ollama** as the LLM inference layer and **OpenClaw** as the agent runtime layer. We demonstrated that modern autonomous systems cannot be understood or evaluated solely based on model capability; instead, their effectiveness emerges from the coordinated interaction between inference, orchestration, memory, and execution components. By formalizing this layered perspective, the paper establishes a clear architectural taxonomy for understanding and designing next-generation **autonomous agents systems**.

The proposed experimental validation provided concrete empirical support for this architectural thesis. Through three progressively complex configurations, the results consistently showed a monotonic improvement in performance ($C1 < C2 < C3$), confirming that capabilities such as tool use, structured data access, and persistent memory are not intrinsic to LLMs alone, but arise from system-level integration. In particular, the full-stack configuration achieved near-perfect task success and deterministic memory recall, highlighting the critical role of persistent memory and runtime orchestration in enabling long-horizon autonomous behavior.

Beyond performance, the study also identified key operational challenges, including security risks, tool misuse, memory integrity, and the complexity of evaluating persistent systems. These findings emphasize that advancing **Agentic AI architecture** requires not only technical innovation but also robust frameworks for safety, governance, and reproducibility.

Looking forward, the integration of scalable multi-agent systems, distributed inference infrastructures, and human-centered design will be essential for deploying reliable and impactful autonomous agents. Ultimately, the future of intelligent systems depends on the development of architectures that are not only efficient and scalable, but also **secure, interpretable, and trustworthy**. This work positions the OpenClaw–Ollama paradigm as a foundational step toward realizing fully autonomous, real-world AI systems grounded in principled architectural design.

Acknowledgements. The publication of the article in OA mode was financially supported by HEAL-Link.

Declarations

Funding

No funding was received to assist with the preparation of this manuscript.

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Ethics Approval

Not applicable.

Consent to Participate

Not applicable.

Consent for Publication

Not applicable.

Data Availability

All data supporting the results and analysis in this article, including experimental datasets, task definitions, and evaluation results, are publicly available in the GitHub repository: <https://github.com/Applied-AI-Research-Lab/OpenClaw-and-Ollama-in-Agent-ic-AI>. The repository includes the complete benchmark task set, configuration files, evaluation scripts, and raw experimental logs.

Code Availability

All source code for the experimental validation, including the evaluation harness, benchmarking tools, and analysis scripts, is publicly available under an open-source license at <https://github.com/Applied-AI-Research-Lab/OpenClaw-and-Ollama-in-Agent-ic-AI>.

Author Contributions

Konstantinos I. Roumeliotis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Visualization. Ranjan Sapkota: Conceptualization, Methodology, Validation, Writing - original draft, Writing - review & editing, Supervision. All authors read and approved the final manuscript.

References

- [1] Chen, Z., Xu, L., Zheng, H., Chen, L., Tolba, A., Zhao, L., Yu, K., Feng, H.: Evolution and prospects of foundation models: From large language models to large multimodal models. *Computers, Materials & Continua* **80**(2) (2024)
- [2] Sarikaya, R.: Path to artificial general intelligence: past, present, and future. *Annual Reviews in Control* **60**, 101021 (2025)
- [3] Liu, X., Wang, J., Yuan, X., Sun, J., Dong, G., Di, P., Wang, W., Wang, D.: Prompting frameworks for large language models: A survey. *ACM Computing Surveys* (2023)
- [4] Zheng, J., Shi, C., Cai, X., Li, Q., Zhang, D., Li, C., Yu, D., Ma, Q.: Lifelong learning of large language model based agents: A roadmap. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2026)
- [5] Sapkota, R., Roumeliotis, K.I., Karkee, M.: Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *Information Fusion*, 103599 (2025)
- [6] Abou Ali, M., Dornaika, F., Charafeddine, J.: Agentic ai: a comprehensive survey of architectures, applications, and future directions. *Artificial Intelligence Review* **59**(1), 11 (2025)
- [7] Alva, L., Pandey, B.: Agentic ai systems in the age of generative models: architectures, cloud scalability, and real-world applications. *Artificial Intelligence Review* (2026)
- [8] Bandi, A., Kongari, B., Naguru, R., Pasnoor, S., Vilipala, S.V.: The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet* **17**(9), 404 (2025)
- [9] Kumar, N., Wei, X., Zhang, H.: Agentic artificial intelligence as a new frontier in information systems: Promise, peril, and research opportunities. *Information & Management*, 104317 (2026)
- [10] Wei, T., Li, T.-W., Liu, Z., Ning, X., Yang, Z., Zou, J., Zeng, Z., Qiu, R., Lin, X., Fu, D., et al.: Agentic reasoning for large language models. *arXiv preprint arXiv:2601.12538* (2026)
- [11] Adabara, I., Sadiq, B.O., Shuaibu, A.N., Danjuma, Y.I., Maninti, V.: Trustworthy agentic ai systems: a cross-layer review of architectures, threat models, and governance strategies for real-world deployment. *F1000Research* **14**(905), 905 (2025)
- [12] Li, X., Wang, S., Zeng, S., Wu, Y., Yang, Y.: A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* **1**(1), 9 (2024)

- [13] He, J., Treude, C., Lo, D.: Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology* **34**(5), 1–30 (2025)
- [14] Rodríguez Quiñones, A.: Privacy-focused llm for local data processing: Implementing ollama and rag to securely query personal files in closed environments. *Universitat Oberta de Catalunya (UOC)* (2025)
- [15] Marcondes, F.S., Gala, A., Magalhães, R., Britto, F., Durães, D., Novais, P.: Using ollama. In: *Natural Language Analytics with Generative Large-Language Models: A Practical Approach with Ollama and Open-Source LLMs*, pp. 23–35. Springer, ??? (2025)
- [16] Weidener, L., Brkić, M., Jovanović, M., Singh, R., Ulgac, E., Meduri, A.: Openclaw, moltbook, and clawdlab: From agent-only social networks to autonomous scientific research. *arXiv preprint arXiv:2602.19810* (2026)
- [17] Borjigin, A., Stadnyk, I., Bilski, B., Hovorov, S., Pidturkina, S.: Execution is the new attack surface: Survivability-aware agentic crypto trading with openclaw-style local executors. *arXiv preprint arXiv:2603.10092* (2026)
- [18] Ying, Z., Yang, X., Wu, S., Song, Y., Qu, Y., Li, H., Li, T., Wang, J., Liu, A., Liu, X.: Uncovering security threats and architecting defenses in autonomous agents: A case study of openclaw. *arXiv preprint arXiv:2603.12644* (2026)
- [19] Deng, X., Zhang, Y., Wu, J., Bai, J., Yi, S., Zou, Z., Xiao, Y., Qiu, R., Ma, J., Chen, J., et al.: Taming openclaw: Security analysis and mitigation of autonomous llm agent threats. *arXiv preprint arXiv:2603.11619* (2026)
- [20] Buyya, R., et al.: Agentic artificial intelligence (ai): Architectures, taxonomies, and evaluation of large language model agents. *arXiv preprint arXiv:2601.12560* (2026)
- [21] Wang, F.Y., Marom, L., Pal, S., Luu, R.K., Lu, W., Berkovich, J.A., Buehler, M.J.: Autonomous agents coordinating distributed discovery through emergent artifact exchange. *arXiv preprint arXiv:2603.14312* (2026)
- [22] Zhang, Z., Valeo, C.: Agentic modelling pipeline: Reproducible rapid stormwater modelling management system with openclaw. *EarthArXiv* (2026)
- [23] Mo, X.: Ironengine: Towards general ai assistant. *arXiv preprint arXiv:2603.08425* (2026)
- [24] Flynn, D.C.: Cognitive integrity security: Securing human-ai interaction in large language model systems. *Authorea Preprints* (2026)

- [25] Holzbauer, F., Schmidt, D., Gegenhuber, G., Schrittwieser, S., Ullrich, J.: Malicious or not: Adding repository context to agent skill classification. arXiv preprint arXiv:2603.16572 (2026)
- [26] Ruan, A.: From logic monopoly to social contract: Separation of power and the institutional foundations for autonomous agent economies. arXiv preprint arXiv:2603.25100 (2026)
- [27] Garg, V.: Designing the mind: How agentic frameworks are shaping the future of ai behavior. *Journal of Computer Science and Technology Studies* **7**(5), 182–193 (2025)
- [28] Olujimi, P.A., Owolawi, P.A., Mogase, R.C., Wyk, E.V.: Agentic ai frameworks in smmes: a systematic literature review of ecosystemic interconnected agents. *AI* **6**(6), 123 (2025)
- [29] Frank, D.-A., Folwarczny, M., Otterbring, T.: Consumer acceptance of high-autonomy ai assistants is driven by perceived benefits in online shopping settings characterized by scarcity. *Psychology & Marketing* **43**(3), 538–555 (2026)
- [30] Kumar, G.: Agentic ai in finance: Building autonomous financial assistants on aws for personalized customer solutions. *International Journal of Innovative Science and Research Technology*, 1502–1512 (2025)
- [31] Luo, L., Hu, J., Zheng, Y., Li, C.: Human vs. ai: Does ai learning assistant enhance students’ innovation behavior? *Education and Information Technologies* **30**(12), 17483–17530 (2025)
- [32] Bowen, G.: Agentic artificial intelligence: Legal and ethical challenges of autonomous systems. *Journal of Digital Technologies and Law* **3**(3), 431–445 (2025)
- [33] Piccoli, G., Rodriguez, J., Mahmoud, A.: Large action models for programmatic orchestration. *Business & Information Systems Engineering*, 1–12 (2025)
- [34] Nygaard, L.H.: Agentic artificial intelligence orchestration and interoperable multi-agent frameworks in enterprise commerce transformation. *European Index Library of European International Journal of Multidisciplinary Research and Management Studies* **6**(01), 108–113 (2026)
- [35] Chopra, A., Bhattacharya, S., Leibo, J.Z., Raskar, R.: Levels of social orchestration for agentic systems. In: *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 (2025)
- [36] Park, S., Jeon, S., Lee, C., Jeon, S., Kim, B.-S., Lee, J.: A survey on inference engines for large language models: Perspectives on optimization and efficiency. *ACM Transactions on Intelligent Systems and Technology* (2025)

- [37] Gao, Z., Xu, Y., Thebault-Spieker, J.: Localbench: Benchmarking llms on county-level local knowledge and reasoning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 38487–38495 (2026)
- [38] Vake, D., Vičić, J., Tošić, A.: Hive: A secure, scalable framework for distributed ollama inference. *SoftwareX* **30**, 102183 (2025)
- [39] Li, X.: A review of prominent paradigms for llm-based agents: Tool use, planning (including rag), and feedback learning. In: Proceedings of the 31st International Conference on Computational Linguistics, pp. 9760–9779 (2025)
- [40] Hao, S., Gu, Y., Ma, H., Hong, J., Wang, Z., Wang, D., Hu, Z.: Reasoning with language model is planning with world model. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 8154–8173 (2023)
- [41] Dou, Z., Zhao, Q., Wan, Z., Zhang, D., Wang, W., Raiyan, T., Chen, B., Pan, Q., Ouyang, Y., Gao, Z., et al.: Plan then action: High-level planning guidance reinforcement learning for llm reasoning. *arXiv preprint arXiv:2510.01833* (2025)
- [42] Salama, R., Cai, J., Yuan, M., Currey, A., Sunkara, M., Zhang, Y., Benajiba, Y.: Meminsight: Autonomous memory augmentation for llm agents. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 33124–33140 (2025)
- [43] Xu, M., Liang, G., Chen, K., Wang, W., Zhou, X., Yang, M., Zhao, T., Zhang, M.: Memory-augmented query reconstruction for llm-based knowledge graph reasoning. In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 24068–24084 (2025)
- [44] Hong, C., He, Q.: Enhancing memory retrieval in generative agents through llm-trained cross attention networks. *Frontiers in Psychology* **16**, 1591618 (2025)
- [45] Wang, C., Zhang, Y., Zhu, F., Zhang, J., Shi, T., Feng, F.: Leveraging memory retrieval to enhance llm-based generative recommendation. In: Companion Proceedings of the ACM on Web Conference 2025, pp. 1346–1350 (2025)
- [46] Luo, H., Liu, Y., Zhang, R., Wang, J., Sun, G., Niyato, D., Yu, H., Xiong, Z., Wang, X., Shen, X.: Toward edge general intelligence with multiple-large language model (multi-llm): architecture, trust, and orchestration. *IEEE Transactions on Cognitive Communications and Networking* (2025)
- [47] Rudra, A., Agrawal, M.: Composable ai stack for intelligent agents: Modular orchestration using context routing, memory, and tools. In: 2025 International Conference on Computer and Applications (ICCA), pp. 1–6 (2025). IEEE

- [48] Andreoletti, D., Rudi, A., Carpanzano, E., Leidi, T.: Privacy-preserving llm inference in practice: A comparative survey of techniques, trade-offs, and deployability. *Cryptology ePrint Archive* (2026)
- [49] Zhang, K., Wang, C., Peng, L., Go, A., Liu, X.: Privacy-preserved llm cascade via cot-enhanced policy learning. In: *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, pp. 955–964 (2026)
- [50] Tanikanti, A., Côté, B., Guo, Y., Chen, L., Saint, N., Chard, R., Raffenetti, K., Thakur, R., Uram, T., Foster, I., *et al.*: First: Federated inference resource scheduling toolkit for scientific ai model access. In: *Proceedings of the SC’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 52–60 (2025)
- [51] Gopee, M.A., Prieto, S.A., Soto, B.G.: Self-hosted multimodal large language models for speech-driven perception and navigation in construction robotics. *Automation in Construction* **183**, 106805 (2026)
- [52] Knoop, J., Holtmann, H.: Private llm inference on consumer blackwell gpus: A practical guide for cost-effective local deployment in smes. *arXiv preprint arXiv:2601.09527* (2026)
- [53] Zhao, Z., Fu, Y., Chai, J., Zhu, Y., Zhao, D.: Meta learning task representation in multiagent reinforcement learning: From global inference to local inference. *IEEE Transactions on Neural Networks and Learning Systems* (2025)
- [54] Kang, H., Li, Z., Yang, X., Xu, W., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., Arora, S.: Thunderagent: A simple, fast and program-aware agentic inference system. *arXiv preprint arXiv:2602.13692* (2026)
- [55] Shan, Z., Xin, J., Zhang, Y., Xu, M.: Don’t let the claw grip your hand: A security analysis and defense framework for openclaw. *arXiv preprint arXiv:2603.10387* (2026)
- [56] Suwansathit, S., Zhang, Y., Gu, G.: A systematic taxonomy of security vulnerabilities in the openclaw ai agent framework. *arXiv preprint arXiv:2603.27517* (2026)
- [57] Yu, J.S., Yao, Y.: Agents and workflows. In: *Intelligent Language Services: Theory and Practice with Large Language Models*, pp. 113–141. Springer, ??? (2026)
- [58] Lee, Y., Park, E.: Toward sustainable agentic ai systems: A survey of architectures and methodologies. *Sustainable Development* (2026)
- [59] Cornacchia, A., Alabdulaal, I., Saghier, I., Mirdad, A., Fayoumi, O., Canini, M.: Between promise and pain: The reality of automating failure analysis in microservices with llms. In: *Proceedings of the 16th ACM SIGOPS Asia-Pacific*

- [60] He, F., Zhu, T., Ye, D., Liu, B., Zhou, W., Yu, P.S.: The emerged security and privacy of llm agent: A survey with case studies. *ACM Computing Surveys* **58**(6), 1–36 (2025)
- [61] Gulyamov, S., Gulyamov, S., Rodionov, A., Khursanov, R., Mekhmonov, K., Babaev, D., Rakhimjonov, A.: Prompt injection attacks in large language models and ai agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms. *Information* **17**(1), 54 (2026)
- [62] Thaiya, M.S., Julia, K., Mbugua, S.: On software modular architecture: Concepts, metrics and trends. *International Journal of Computer & Organization Trends* **12**(1), 3–10 (2022)
- [63] Mari, Eila: The impact of maintainability on component-based software systems. In: 2003 Proceedings 29th Euromicro Conference, pp. 25–32 (2003). IEEE
- [64] Zhang, R., Liu, G., Liu, Y., Zhao, C., Wang, J., Xu, Y., Niyato, D., Kang, J., Li, Y., Mao, S., *et al.*: Toward edge general intelligence with agentic ai and agentification: Concepts, technologies, and future directions. *IEEE Communications Surveys & Tutorials* **28**, 4285–4318 (2026)
- [65] Brohi, S., Mastoi, Q.-u.-a., Jhanjhi, N., Pillai, T.R.: A research landscape of agentic ai and large language models: Applications, challenges and future directions. *Algorithms* **18**(8), 499 (2025)
- [66] Rohitha, K.P.: *Agentic AI-Building Autonomous Intelligent Systems for Enterprise Applications*. AGPH Books| AG Publishing House, ??? (2025)
- [67] Biswas, B., Sarkar, S.: Responsible agentic artificial intelligence governance: Risk, safety, and ethical challenges in autonomous systems. *International Journal of Applied Resilience and Sustainability* **2**(2), 142–167 (2026)
- [68] Janakiraman, A.: Governance and accountability frameworks for ai agents. *Synergia: A Journal of Multidisciplinary Innovation* **7**(7) (2025)
- [69] Sarin, S., Singh, L., Sarmah, B., Mehta, D.: Memoria: A scalable agentic memory framework for personalized conversational ai. In: 2025 5th International Conference on AI-ML-Systems (AIMLSystems), pp. 32–39 (2025). IEEE
- [70] Kang, J., Ji, M., Zhao, Z., Bai, T.: Memory os of ai agent. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 25972–25981 (2025)
- [71] Lesser, V.R.: Reflections on the nature of multi-agent coordination and its implications for an agent architecture. *Autonomous agents and multi-agent*

systems **1**(1), 89–111 (1998)

- [72] Ahmed, I., Syed, M.A., Maaruf, M., Khalid, M.: Distributed computing in multi-agent systems: a survey of decentralized machine learning approaches. *Computing* **107**(1), 2 (2025)
- [73] Álvarez-López, C., González-Briones, A., Li, T.: Explainable ai and multi-agent systems for energy management in iot-edge environments: A state of the art review. *Electronics* **15**(2), 385 (2026)
- [74] Coelho, R.M., Lima, V.M.A.: Adaptive load balancing for distributed llm inference using ollama. *Fundação Universidade Federal de Mato Grosso do Sul* (2025)
- [75] Liu, Z., Wang, Q., Lu, H., Wang, Y.: Feasibility and usability practice on local hosting open source large language models (llms) including llama 3.2 vision 90b in multi-functional agentic artificial intelligence (ai) system to drive service for design in the latest affordable small personal computer (pc) system. In: *International Conference on Human-Computer Interaction*, pp. 261–279 (2025). Springer
- [76] Chen, S., Wang, Q., Yu, G., Wang, X., Zhu, L.: Clawed and dangerous: Can we trust open agentic systems? *arXiv preprint arXiv:2603.26221* (2026)
- [77] Wang, Y., Chen, X., Jin, X., Wang, M., Yang, L.: Openclaw-rl: Train any agent simply by talking. *arXiv preprint arXiv:2603.10165* (2026)
- [78] Wang, H., Gong, J., Zhang, H., Xu, J., Wang, Z.: Ai agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126* (2025)
- [79] Yang, J., Liu, X., Lv, W., Deng, K., Guo, S., Jing, L., Li, Y., Liu, S., Luo, X., Luo, Y., et al.: From code foundation models to agents and applications: A comprehensive survey and practical guide to code intelligence. *arXiv preprint arXiv:2511.18538* (2025)
- [80] Hermawan, R., Bucaioni, A., Enoiu, E., Afzal, W.: Benchmarking large language models for root cause analysis in train control software testing. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 648–657 (2025). IEEE
- [81] Srivastava, G., Hussain, A., Bi, Z., Roy, S., Pitre, P., Lu, M., Ziyadi, M., Wang, X.: Beyondbench: Benchmark-free evaluation of reasoning in language models. *arXiv preprint arXiv:2509.24210* (2025)
- [82] Ji, Y., Li, J., Xiang, Y., Ye, H., Wu, K., Yao, K., Xu, J., Mo, L., Zhang, M.: A survey of test-time compute: From intuitive inference to deliberate reasoning.

arXiv preprint arXiv:2501.02497 (2025)

- [83] Yang, X., Li, L., Zhou, H., Zhu, T., Qu, X., Fan, Y., Wei, Q., Ye, R., Kang, L., Qin, Y., et al.: Toward efficient agents: Memory, tool learning, and planning. arXiv preprint arXiv:2601.14192 (2026)
- [84] Vishnubhatla, S.: Agentic intelligence in information management systems: A framework for autonomous decision workflows. *European Journal of Advances in Engineering and Technology* **12**(6), 108–115 (2025)
- [85] Chkirkbene, Z., Hamila, R., Gouisssem, A., Devrim, U.: Large language models (llm) in industry: A survey of applications, challenges, and trends. In: 2024 IEEE 21st International Conference on Smart Communities: Improving Quality of Life Using AI, Robotics and IoT (HONET), pp. 229–234 (2024). IEEE
- [86] Chen, X., Gao, C., Chen, C., Zhang, G., Liu, Y.: An empirical study on challenges for llm application developers. *ACM Transactions on Software Engineering and Methodology* **34**(7), 1–37 (2025)
- [87] Gao, M., Hu, X., Yin, X., Ruan, J., Pu, X., Wan, X.: Llm-based nlg evaluation: Current status and challenges. *Computational Linguistics* **51**(2), 661–687 (2025)
- [88] Yi, Z., Ouyang, J., Xu, Z., Liu, Y., Liao, T., Luo, H., Shen, Y.: A survey on recent advances in llm-based multi-turn dialogue systems. *ACM Computing Surveys* **58**(6), 1–38 (2025)
- [89] Hua, Y., Li, K., Wang, R., Li, Y., Wang, G., Yan, Y.: Integration of dynamic knowledge and llm for adaptive human-robot collaborative assembly solution generation. *Advanced Engineering Informatics* **68**, 103613 (2025)
- [90] Sibai, N., Ahmed, Y., Sibae, S., AlHalawani, S., Ammar, A., Boulila, W.: The path ahead for agentic ai: Challenges and opportunities. arXiv preprint arXiv:2601.02749 (2026)
- [91] Kampik, T., Mansour, A., Boissier, O., Kirrane, S., Padget, J., Payne, T.R., Singh, M.P., Tamma, V., Zimmermann, A.: Governance of autonomous agents on the web: Challenges and opportunities. *ACM Transactions on Internet Technology* **22**(4), 1–31 (2022)
- [92] Gupta, A., Remella, S.: Agentic ai with cybersecurity: How to focus on risk analysis via the mcp (model–control–policy) model. *International Journal of Emerging Trends in Computer Science and Information Technology* **7**(1), 8–15 (2026)
- [93] Perwej, Y.: Machine learning driven predictive modelling for intelligent data-centric applications. *Machine Learning* **36**(1), 678–683 (2026)

- [94] Anakabe, O.G., Cocola, R., Aceta, C.: Adding determinism to a dialogue agent for a robotic environment. In: *Proceedings of the 16th International Workshop on Spoken Dialogue System Technology*, pp. 253–261 (2026)
- [95] Zhu, Y., Zhu, S.-C.: Tool-use. In: *Computer Vision: Cognitive Models for Visual Commonsense*, pp. 139–180. Springer, ??? (2026)
- [96] Yang, W., Qiu, H., Zhang, B., Li, C., Huang, Z., Feng, X., Yu, R., Dong, J.: When openclaw meets hospital: Toward an agentic operating system for dynamic clinical workflows. *arXiv preprint arXiv:2603.11721* (2026)
- [97] Chen, H., Liu, X., He, B., Liu, X.: From inference routing to agent orchestration: Declarative policy compilation with cross-layer verification. *arXiv preprint arXiv:2603.27299* (2026)
- [98] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: React: Synergizing reasoning and acting in language models. In: *The Eleventh International Conference on Learning Representations* (2022)
- [99] Wang, C., Bao, W., Gao, S., Xu, B., Tian, Y., Rawat, Y.S., Ge, Y., Shang, Y.: Vla-thinker: Boosting vision-language-action models through thinking-with-image reasoning. *arXiv preprint arXiv:2603.14523* (2026)
- [100] Li, H.: General framework of ai agents. *Journal of Computer Science and Technology*, 1–13 (2026)
- [101] Masterman, T., Besen, S., Sawtell, M., Chao, A.: The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584* (2024)
- [102] Patil, S.G., Mao, H., Yan, F., Ji, C.C.-J., Suresh, V., Stoica, I., Gonzalez, J.E.: The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In: *Forty-second International Conference on Machine Learning* (2025)
- [103] Song, Y., Xu, F.F., Zhou, S., Neubig, G.: Beyond browsing: Api-based web agents. In: *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 11066–11085 (2025)
- [104] Pang, X., Tang, S., Ye, R., Du, Y., Du, Y., Chen, S.: Browsemaster: Towards scalable web browsing via tool-augmented programmatic agent pair. *arXiv preprint arXiv:2508.09129* (2025)
- [105] Guo, C., Liu, X., Xie, C., Zhou, A., Zeng, Y., Lin, Z., Song, D., Li, B.: Redcode: Risky code execution and generation benchmark for code agents. *Advances in Neural Information Processing Systems* **37**, 106190–106236 (2024)

- [106] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H.: Executable code actions elicit better llm agents. In: Forty-first International Conference on Machine Learning (2024)
- [107] Shorten, C., Pierse, C., Smith, T.B., D’Oosterlinck, K., Celik, T., Cardenas, E., Monigatti, L., Hasan, M.S., Schmuhl, E., Williams, D., et al.: Querying databases with function calling. arXiv preprint arXiv:2502.00032 (2025)
- [108] Zhang, Y.: Agent-as-tool: A study on the hierarchical decision making with reinforcement learning. arXiv preprint arXiv:2507.01489 (2025)
- [109] Elder, B., Murthi, A., Kang, J., Naik, A., Basu, K., Kate, K., Contractor, D.: Live api-bench: 2500+ live apis for testing multi-step tool calling. In: Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 3092–3124 (2026)
- [110] Ni, X., Jian, H., Wang, Q., Shah, V.C., Hong, P.: Doc2agent: Scalable generation of tool-using agents from api documentation. arXiv preprint arXiv:2506.19998 (2025)
- [111] Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., Wen, J.-R.: A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* **43**(6), 1–47 (2025)
- [112] Iannino, V., Mocci, C., Vannocci, M., Colla, V., Caputo, A., Ferraris, F.: An event-driven agent-based simulation model for industrial processes. *Applied Sciences* **10**(12), 4343 (2020)
- [113] Calvaresi, D., Dicente Cid, Y., Marinoni, M., Dragoni, A.F., Najjar, A., Schumacher, M.: Real-time multi-agent systems: rationality, formal model, and empirical results. *Autonomous agents and multi-agent systems* **35**(1), 12 (2021)
- [114] Cao, S., Chang, Z., Li, C., Li, H., Fu, L., Tang, J.: The auton agentic ai framework. arXiv preprint arXiv:2602.23720 (2026)
- [115] Park, G., Lee, S., Park, Y.: Minimizing response latency in llm-based agent systems: A comprehensive survey. *IEEE Access* (2026)
- [116] Li, Z., Meng, X., He, X., Zhang, Y., Yin, W.: Large-scale model-enhanced vision-language navigation: Recent advances, practical applications, and future challenges. *Sensors* **26**(7), 2022 (2026)
- [117] Arora, S., Hastings, J.: Securing agentic ai systems-a multilayer security framework. In: 2025 5th International Conference on Robotics, Automation, and Artificial Intelligence (RAAI), pp. 509–514 (2025). IEEE

- [118] Agarwal, A., Nene, M.J.: A five-layer framework for ai governance: integrating regulation, standards, and certification. *Transforming Government: People, Process and Policy* **19**(3), 535–555 (2025)
- [119] Kiasari, M., Aly, H.: Agentic artificial intelligence for smart grids: A comprehensive review of autonomous, safe, and explainable control frameworks. *Energies* **19**(3), 617 (2026)
- [120] Jaggavarapu, M.K.R.: The evolution of agentic ai: architecture and workflows for autonomous systems. *Journal Of Multidisciplinary* **5**(7), 418–427 (2025)
- [121] Pati, A.K.: Agentic ai: a comprehensive survey of technologies, applications, and societal implications. *IEEE Access* (2025)
- [122] Rosic, M., Sumic, D., Males, L.: Semantic interoperability of multi-agent systems in autonomous maritime domains. *Electronics* **14**(13), 2630 (2025)
- [123] Xu, W., Chen, J., Zheng, P., Yi, X., Tian, T., Zhu, W., Wan, Q., Wang, H., Fan, Y., Su, Q., et al.: Deploying foundation model powered agent services: A survey. *IEEE Communications Surveys & Tutorials* (2025)
- [124] Yazdanian, P., Liu, Y., Li, Z.: A comparative study towards designing a hybrid architecture of microservices and llm-based multi-agent systems. In: 2025 32nd Asia-Pacific Software Engineering Conference (APSEC), pp. 761–772 (2025). IEEE
- [125] Astobiza, A.M.: Do ai agents trump human agency? *Discover Artificial Intelligence* **5**(1), 348 (2025)
- [126] Sun, M., Hong, F., Zhang, W.: Sophia: A persistent agent framework of artificial life. *arXiv preprint arXiv:2512.18202* (2025)
- [127] Deng, S., Zhao, H., Wang, Z., Cheng, G., Chen, P., Qian, W., Ling, Z., Yin, J., Zomaya, A.Y., Dustdar, S.: Agentic services computing. *arXiv preprint arXiv:2509.24380* (2025)
- [128] Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., et al.: Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564* (2025)
- [129] Maragheh, R.Y., Deldjoo, Y.: The future is agentic: Definitions, perspectives, and open challenges of multi-agent recommender systems. *arXiv preprint arXiv:2507.02097* (2025)
- [130] Lu, P., Chen, B., Liu, S., Thapa, R., Boen, J., Zou, J.: Octotools: An agentic framework with extensible tools for complex reasoning. *arXiv preprint arXiv:2502.11271* (2025)
- [131] Galster, M., Mohsenimofidi, S., Lulla, J.L., Abubakar, M.A., Treude, C., Baltes,

- S.: Configuring agentic ai coding tools: An exploratory study. arXiv preprint arXiv:2602.14690 (2026)
- [132] Radanliev, P.: Transparent by design: Ensuring safety in agentic ai through decision traceability. In: Ethical AI and Data Science, pp. 1–20. Auerbach Publications, ??? (2026)
 - [133] Hmimou, Y., Tabaa, M., Khiat, A., Hidila, Z.: A dataset capturing decision processes, tool interactions and provenance links in autonomous ai agents. *Data* **11**(4), 66 (2026)
 - [134] Fournier, F., Limonad, L., David, Y.: Agentic ai process observability: Discovering behavioral variability. arXiv preprint arXiv:2505.20127 (2025)
 - [135] AlSayyad, A., Huang, K.Y., Pal, R.: Agenttrace: A structured logging framework for agent system observability. In: LLM-based Multi-Agent Systems: Towards Responsible, Reliable, and Scalable Agentic Systems (2026)
 - [136] Moshkovich, D., Mulian, H., Zeltyn, S., Eder, N., Skarbovsky, I., Abitbol, R.: Beyond black-box benchmarking: Observability, analytics, and optimization of agentic systems. arXiv preprint arXiv:2503.06745 (2025)
 - [137] Raheem, T., Hossain, G.: Agentic ai systems: Opportunities, challenges, and trustworthiness. In: 2025 IEEE International Conference on Electro Information Technology (eIT), pp. 618–624 (2025). IEEE
 - [138] Sharma, A.: Agentic ai security: Threat modeling+ controls for ai agents (permissions, tool-use constraints, auditability, kill-switch design). *SAMRIDDHI: A Journal of Physical Sciences, Engineering and Technology* **17**(02), 47–60 (2025)
 - [139] Alqithami, S.: Autonomous agents on blockchains: Standards, execution models, and trust boundaries. arXiv preprint arXiv:2601.04583 (2026)
 - [140] Kumar, S.N.P.: Regulating autonomous ai agents: Prospects, hazards, and policy structures. *Journal of Computer Science and Technology Studies* **7**(10), 393–399 (2025)
 - [141] Leo, M., Tan, F., Miao, T., Anand, G.: From threat to trust: assessing security risks of agentic ai systems: M. leo et al. *International Journal of Information Security* **25**(1), 23 (2026)
 - [142] Kim, J., Shin, B., Chung, J., Rhu, M.: The cost of dynamic reasoning: Demystifying ai agents and test-time scaling from an ai infrastructure perspective. In: 2026 IEEE International Symposium on High Performance Computer Architecture (HPCA), pp. 1–16 (2026). IEEE
 - [143] Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building

production-ready ai agents with scalable long-term memory. arXiv preprint arXiv:2504.19413 (2025)

- [144] Wang, X., Li, Q., Jia, W.: Cognitive edge computing: A comprehensive survey on optimizing large models and ai agents for pervasive deployment. arXiv preprint arXiv:2501.03265 (2025)
- [145] Dhevi, B.L., Neerumalla, S., Pandi, V.S., Dinesh, M., Giri, J., Sabri, M.O.: Effective cloud computing architectures for complex systems: Scalable data processing and real-time decision making. In: 2025 International Conference on Networks and Cryptology (NETCRYPT), pp. 1142–1147 (2025). IEEE
- [146] Hossain, I.: Micro-agent collectives: A scalable architecture for distributed tool-oriented reasoning. *Multidisciplinary Innovations & Research Analysis* **6**(4), 42–63 (2025)
- [147] Dai, F., Hossain, M.A., Wang, Y.: State of the art in parallel and distributed systems: Emerging trends and challenges. *Electronics* **14**(4), 677 (2025)
- [148] Zhao, W., Yu, Y., Mao, B., Kato, N.: A survey on computing power networks: Architecture, resource allocation, and ai services enablement. *IEEE Transactions on Network Science and Engineering* (2026)
- [149] Mishra, S., Niroula, S., Yadav, U., Thakur, D., Gyawali, S., Gaire, S.: Sok: Agentic retrieval-augmented generation (rag): Taxonomy, architectures, evaluation, and research directions. arXiv preprint arXiv:2603.07379 (2026)
- [150] Lewis, E., Parker, B., Johnson, A.: Quantifying the stability of distributed enterprise architectures through multi-agent system simulations. *Journal of Innovation in Governance and Business Practices* **2**, 122–151 (2026)
- [151] Raeispour, M., Yan, S., Meegahapola, L., Yu, X.: Cyber-physical security of virtual power plants: A survey. *IEEE Open Journal of the Industrial Electronics Society* (2026)
- [152] Maharramova, A., Abbasova, A., Imanova, S., Hakhverdiyev, T., Rustamovna, A.K.: Artificial intelligence (ai) agents. *Reliability: Theory & Applications* **20**(SI 10 (88)), 166–175 (2025)
- [153] Shahout, R., Tirmazi, H., Yu, M., Mitzenmacher, M.: Orla: A library for serving llm-based multi-agent systems. arXiv preprint arXiv:2603.13605 (2026)
- [154] Wu, C., Zhong, H., Jiao, Z., Yuan, G., Hu, S., Zhang, J., Sun, R., Peng, C., Xiao, Y.: Development of a simplified multi-agent system for fault diagnosis. In: 2025 Global Reliability and Prognostics and Health Management Conference (PHM-Xian), pp. 1–8 (2025). IEEE

- [155] Huang, D., Wang, Z.: Llms at the edge: Performance and efficiency evaluation with ollama on diverse hardware. In: 2025 International Joint Conference on Neural Networks (IJCNN), pp. 1–8 (2025). IEEE
- [156] Rajesh, V., Jodhpurkar, O., Anbuselvan, P., Singh, M., Jallepali, A., Godbole, S., Sharma, P.K., Shrivastava, H.: Production-grade local llm inference on apple silicon: A comparative study of mlx, mlc-llm, ollama, llama. cpp, and pytorch mps. arXiv preprint arXiv:2511.05502 (2025)
- [157] Li, F.: Openclaw prism: A zero-fork, defense-in-depth runtime security layer for tool-augmented llm agents. arXiv preprint arXiv:2603.11853 (2026)
- [158] Ke, Z., Jiao, F., Ming, Y., Nguyen, X.-P., Xu, A., Long, D.X., Li, M., Qin, C., Wang, P., Savarese, S., et al.: A survey of frontiers in llm reasoning: Inference scaling, learning to reason, and agentic systems. arXiv preprint arXiv:2504.09037 (2025)
- [159] Menezes, L.C.M.d.M.: An ai-driven architecture for intelligent log observability. Idp (2026)
- [160] Rosas, E., Prades, J.: Context-aware ai assistants for hands-on and interactive computing education. In: EDULEARN25 Proceedings, pp. 5415–5422 (2025). IATED
- [161] Vega-Márquez, B.: Context-aware ai agents for clinical dialogue assistance through large language models. In: Artificial Intelligence in Biomedicine: First Conference of the Spanish Society of Artificial Intelligence in Biomedicine, CIA-Biomed 2025, Seville, Spain, October 23–24, 2025, Proceedings, p. 163 (2026). Springer Nature
- [162] Naranjo Pozas, R., Doblado Mendoza, P., Vega Márquez, B.: Context-aware ai agents for clinical dialogue assistance through large language models. Springer (2026)
- [163] Strehlow, R.K., Küster, T., Kupke, O.F., Kurps, B.L., Sivrikaya, F., Albayrak, S.: Sage: Tool-augmented llm task solving strategies in scalable multi-agent environments. arXiv preprint arXiv:2601.09750 (2026)
- [164] Liu, H.-I., Galindo, M., Xie, H., Wong, L.-K., Shuai, H.-H., Li, Y.-H., Cheng, W.-H.: Lightweight deep learning for resource-constrained environments: A survey. ACM Computing Surveys **56**(10), 1–42 (2024)
- [165] Ngo, D., Park, H.-C., Kang, B.: Edge intelligence: A review of deep neural network inference in resource-limited environments. Electronics **14**(12), 2495 (2025)
- [166] Zheng, Y., Chen, Y., Qian, B., Shi, X., Shu, Y., Chen, J.: A review on edge large language models: Design, execution, and applications. ACM Computing Surveys

- [167] Choi, H., Jeong, J.: Domain-specific manufacturing analytics framework: an integrated architecture with retrieval-augmented generation and ollama-based models for manufacturing execution systems environments. *Processes* **13**(3), 670 (2025)
- [168] Gong, W.G.: Structured prompt language: Declarative context management for llms. arXiv preprint arXiv:2602.21257 (2026)
- [169] Muraru, M.-M., Evstafev, E., Iantovics, L.B.: Study of small and medium-scale open-source language models in human-like logic and comprehension tasks. *Procedia Computer Science* **270**, 4776–4785 (2025)
- [170] Bandara, E., Gore, R., Yarlagadda, A., Clayton, A.H., Samuel, P., Rhea, C.K., Shetty, S.: Standardization of psychiatric diagnoses—role of fine-tuned llm consortium and openai-gpt-oss reasoning llm enabled decision support system. arXiv preprint arXiv:2510.25588 (2025)
- [171] Bandara, E., Gore, R., Shetty, S., Mukkamala, R., Rhea, C.K., Samulski, B.S., Hass, A., Yarlagadda, A., Kaushik, S., De Silva, M., *et al.*: Standardization of neuromuscular reflex analysis—role of fine-tuned vision-language model consortium and openai gpt-oss reasoning llm-enabled decision support system. *Biomechanics* **6**(1), 23 (2026)
- [172] Chen, E., Guan, C., Elshafiey, A., Zhao, Z., Zekeri, J., Shaibu, A.E., Prince, E.O., Wu, C.-J.: When openclaw agents learn from each other: Insights from emergent ai agent communities for human-ai partnership in education. arXiv preprint arXiv:2603.16663 (2026)
- [173] Venerito, V., Lopalco, G., Iannone, F., Gupta, L.: Agentic ai in rheumatology. *The Lancet Rheumatology* (2026)
- [174] Manik, M.M.H., Wang, G.: Openclaw agents on moltbook: Risky instruction sharing and norm enforcement in an agent-only social network. arXiv preprint arXiv:2602.02625 (2026)
- [175] Huang, K.: *Agentic AI*. Springer, ??? (2025)
- [176] Xia, P., Chen, J., Yang, X., Tu, H., Liu, J., Xiong, K., Han, S., Qiu, S., Ji, H., Zhou, Y., *et al.*: Metaclaw: Just talk—an agent that meta-learns and evolves in the wild. arXiv preprint arXiv:2603.17187 (2026)
- [177] Nie, X., Guo, Z., Yang, K., Zheng, Z., Ge, B., Pan, S., Chen, Z., Xiang, Y., Zhang, Y., Liu, W., *et al.*: Synergy: A next-generation general-purpose agent for open agentic web. arXiv preprint arXiv:2603.28428 (2026)

- [178] Kelley, B.N., Williams, R.A., Holland, J.L., Schnarr, O.C., Allen, B.D.: A persistent simulation environment for autonomous systems. In: 2018 Aviation Technology, Integration, and Operations Conference, p. 4015 (2018)
- [179] Liang, J., Li, H., Li, C., Zhou, J., Jiang, S., Wang, Z., Ji, C., Zhu, Z., Liu, R., Ren, T., et al.: Ai meets brain: Memory systems from cognitive neuroscience to autonomous agents. arXiv preprint arXiv:2512.23343 (2025)
- [180] Chen, J., Chen, Z., Du, C., He, M., He, W., Li, H., Li, Q., Liu, Z., Ma, H., Pan, X., et al.: Streamingclaw technical report. arXiv preprint arXiv:2603.22120 (2026)
- [181] Zhang, Y., Zhao, S., Zhang, J., Deng, G., Zhang, J., Liu, X., Xiao, C., Zhang, T.: Mind your heartbeat! claw background execution inherently enables silent memory pollution. arXiv preprint arXiv:2603.23064 (2026)
- [182] Li, Z., Li, W., Li, X.: Defensible design for openclaw: Securing autonomous tool-invoking agents. arXiv preprint arXiv:2603.13151 (2026)
- [183] Ding, M., Huang, C., Hu, Y., Li, Y., Lu, Z., Yu, X., Zhang, D., Zhai, W., Zhu, T., Gu, Q., et al.: Automating computational chemistry workflows via openclaw and domain-specific skills. arXiv preprint arXiv:2603.25522 (2026)
- [184] Roman, A., Roman, J.: Orchestral ai: A framework for agent orchestration. arXiv preprint arXiv:2601.02577 (2026)
- [185] Xu, H., Li, C., Ma, X., Ou, X., Zhang, Z., He, T., Liu, X., Wang, Z., Liang, J., Chu, Z., et al.: The evolution of tool use in llm agents: From single-tool call to multi-tool orchestration. arXiv preprint arXiv:2603.22862 (2026)
- [186] Dakov, S., Dakova, M.: Event-driven data orchestration: A modular approach for high-volume real-time processing. *Engineering Proceedings* **100**(1), 48 (2025)
- [187] Alibaba Cloud Qwen Team: Qwen3.5:4B. <https://ollama.com/library/qwen3.5:4b>. Accessed: 2026-04-07 (2026)
- [188] Google DeepMind: Gemma 4: E4B. <https://ollama.com/library/gemma4:e4b>. Accessed: 2026-04-07 (2026)